COMPLETE BACKEND GUIDE · BUILT FOR FLUTTER APPS

# Flutter App Backend Blueprint

A secure, fast, crash-resistant and easy-to-manage backend, designed to run on free tools.



Secure | No crash | No delay | Easy to manage | Free tier

## Inside this guide

- 8 flowcharts: architecture, request flow, login, token refresh, self-healing, cache, jobs, CI/CD
- Every API explained: method, path, who can call it, and what is checked
- Security for the server AND the Flutter app (OWASP API Top 10 mapped)
- Crash prevention, speed rules, backups, monitoring and alerts
- Ready-to-use Docker, Caddy, Fastify, backup, deploy, k6 and Dio code
- A master checklist so nothing is forgotten

Version 1.0 · November 2026

Free-tier limits change often. Always confirm limits on each provider's own pricing page before you commit.

# Contents


**1. Read this first: what you will get . . . 5**

1.1 The honest truth about “no crash” and “no delay” . . . 5

1.2 Free-hosting traps that cause delay or crashes . . . 5

1.3 How to read this guide . . . 5

**2. The big picture (architecture) . . . 6**

**3. The free tool stack . . . 8**

3.1 RAM and disk budget for one free VM . . . 8

**4. The life of one request . . . 10**

**5. Every API, explained . . . 13**

5.1 Rules that apply to every API . . . 13

5.2 Group A: System and app configuration . . . 14

5.3 Group B: Authentication . . . 14

5.4 Group C: User profile and devices . . . 15

5.5 Group D: Your main data (example name: items) . . . 15

5.6 Group E: Jobs and notifications . . . 16

5.7 Group F: Payments (only if your app sells something) . . . 16

5.8 Group G: Admin (separate role, extra protection) . . . 16

5.9 What status code means, and what the Flutter app should do . . . 17

**6. Login, tokens, and sessions . . . 18**

6.1 Token rules . . . 19

6.2 Rate limits (starting values) . . . 20

**7. Security: no weak spots . . . 21**

7.1 The seven layers . . . 21

7.2 OWASP API Security Top 10 (2023): risk by risk . . . 21

7.3 Server hardening, step by step . . . 22

7.4 Security headers and CORS . . . 22

7.5 The Flutter app's half of the security . . . 22

7.6 Data protection and privacy . . . 23

7.7 Free security testing tools . . . 23

7.8 If something goes wrong: incident plan . . . 23

**8. No crash: reliability and self-healing . . . 25**

8.1 Why backends crash, and how we prevent each cause . . . 25

8.2 The reliability rules . . . 26

8.3 Self-healing flow . . . 26

8.4 Backups and disaster recovery . . . 27

**9. No delay: speed rules . . . 29**
9.1 Where the time goes . . . 29
9.2 Techniques, from biggest win to smallest . . . 29
9.3 Cache-aside, step by step . . . 29
9.4 Heavy work without waiting . . . 30
9.5 Flutter-side speed tips . . . 31
9.6 Load test before launch . . . 31

**10. Easy to manage . . . 32**
10.1 Project layout . . . 32
10.2 Three layers inside each module . . . 32
10.3 One command for everything . . . 32
10.4 Environments . . . 32
10.5 Automatic release pipeline . . . 33
10.6 Logging standard . . . 34
10.7 Versioning without breaking old apps . . . 34
10.8 Documentation people really use . . . 34

**11. Ready-to-use code and config . . . 35**
11.1 docker-compose.yml . . . 35
11.2 Dockerfile for the API . . . 36
11.3 Caddyfile . . . 36
11.4 Fastify server: security, health, errors, graceful shutdown . . . 37
11.5 Backup script (cron: 15 2 * * *) . . . 37
11.6 Deploy script with automatic rollback . . . 38
11.7 GitHub Actions: test, scan, deploy . . . 38
11.8 k6 load test . . . 39
11.9 Flutter: Dio interceptor with safe token refresh . . . 39

**12. Database design that stays fast and safe . . . 40**
12.1 Rules for every table . . . 40
12.2 Core tables . . . 40
12.3 PostgreSQL starting settings (for about 3 GB of RAM) . . . 40

**13. Monitoring, alerts, and runbooks . . . 41**
13.1 Alerts (send to Telegram or email) . . . 41
13.2 Dashboards to build . . . 41
13.3 Runbook: symptom, check, fix . . . 42

**14. When free is not enough: the upgrade path . . . 43**

**15. Master checklist: check every single thing . . . 44**
A. Security . . . 44
B. Reliability (no crash) . . . 45

C. Speed (no delay) . . . 45
D. Easy to manage . . . 46
E. Flutter app . . . 46
F. Launch day . . . 47

**16. A 30-day plan . . . 47**

Sources and notes . . . 47

**Quick start**
In a hurry? Read Section 1 (honest goals), look at Figure 1 (architecture) and Figure 2 (one request), then jump to Section 15 (master checklist). Everything else explains the “why” and the “how”.

# 1. Read this first: what you will get

You asked for a backend for your Flutter app that is **secure**, **does not crash**, **has no delay**, is **easy to manage**, and uses **free tools**. This guide gives you the full design, 8 flowcharts, every API, the security rules, and a master checklist so you can verify every single thing.

## 1.1 The honest truth about “no crash” and “no delay”

**Zero crashes and zero delay do not exist.**
Even the biggest companies have outages. The real goal has four parts: (1) crash very rarely, (2) heal itself in seconds when something breaks, (3) answer fast for normal users, and (4) never lose user data. This blueprint is designed for exactly these four goals.

| Goal | Target in real numbers | How this guide reaches it |
|---|---|---|
| **Secure** | Everything over HTTPS. No known holes from the OWASP API Top 10. No secrets inside code or inside the app. | Section 7 and the checklist |
| **No crash** | 99.9% uptime per month (at most about 43 minutes of downtime in 30 days). | 2 API containers, auto-restart, health checks, auto-rollback (Section 8) |
| **No delay** | 95% of API calls answered in under 300 ms. Cache hits under 20 ms. No cold starts. | Always-on server, Redis cache, DB indexes, job queue (Section 9) |
| **Easy to manage** | One command to deploy. One dashboard. Alerts on your phone. Nightly backups. | Section 10 and Section 13 |
| **Free** | $0 server cost to start, using free tiers only. | Section 3 (and the upgrade path in Section 14) |

## 1.2 Free-hosting traps that cause delay or crashes

**Trap 1: sleeping free servers**
Some free hosts put your API to sleep when nobody uses it. For example, Render's free web services spin down after 15 minutes without traffic, and the next request waits about 30 to 60 seconds while the server wakes up. A Flutter user would see a frozen screen. Do not use a sleeping service as your main API.

**Trap 2: the free allowance changes**
Oracle Cloud's Always Free virtual machine is the best free always-on server. But Oracle reduced the free Arm allowance to about 2 CPUs and 12 GB RAM in June 2026 (many older guides still say 4 CPUs and 24 GB). Idle free VMs can also be reclaimed, and capacity is sometimes hard to get. Check Oracle's current Always Free page before you start.

**Trap 3: edge-worker limits**
Cloudflare Workers is very fast, but its free plan allows 100,000 requests per day and 10 ms of CPU time per request. That is fine for a small app, but a busy app will hit the wall and requests will fail.

**The smart plan**
Start free on an always-on VM. When real users arrive, pay a small amount ($5 to $10 per month) for a second server or a paid tier. That is the cheapest way to get real reliability. See Section 14.

## 1.3 How to read this guide

- Flowcharts are numbered Figure 1 to Figure 8. Read them first; the text explains every box.
- Section 5 lists every API: method, path, who can call it, and what is checked.
- Code samples are ready to copy. Replace `yourapp.com` and the example names with your own.
- Section 15 is the master checklist. Tick every box before you launch.

# 2. The big picture (architecture)

Every request travels through layers. Each layer has one job, and each layer protects the next one. If one layer fails, the other layers still protect your data.

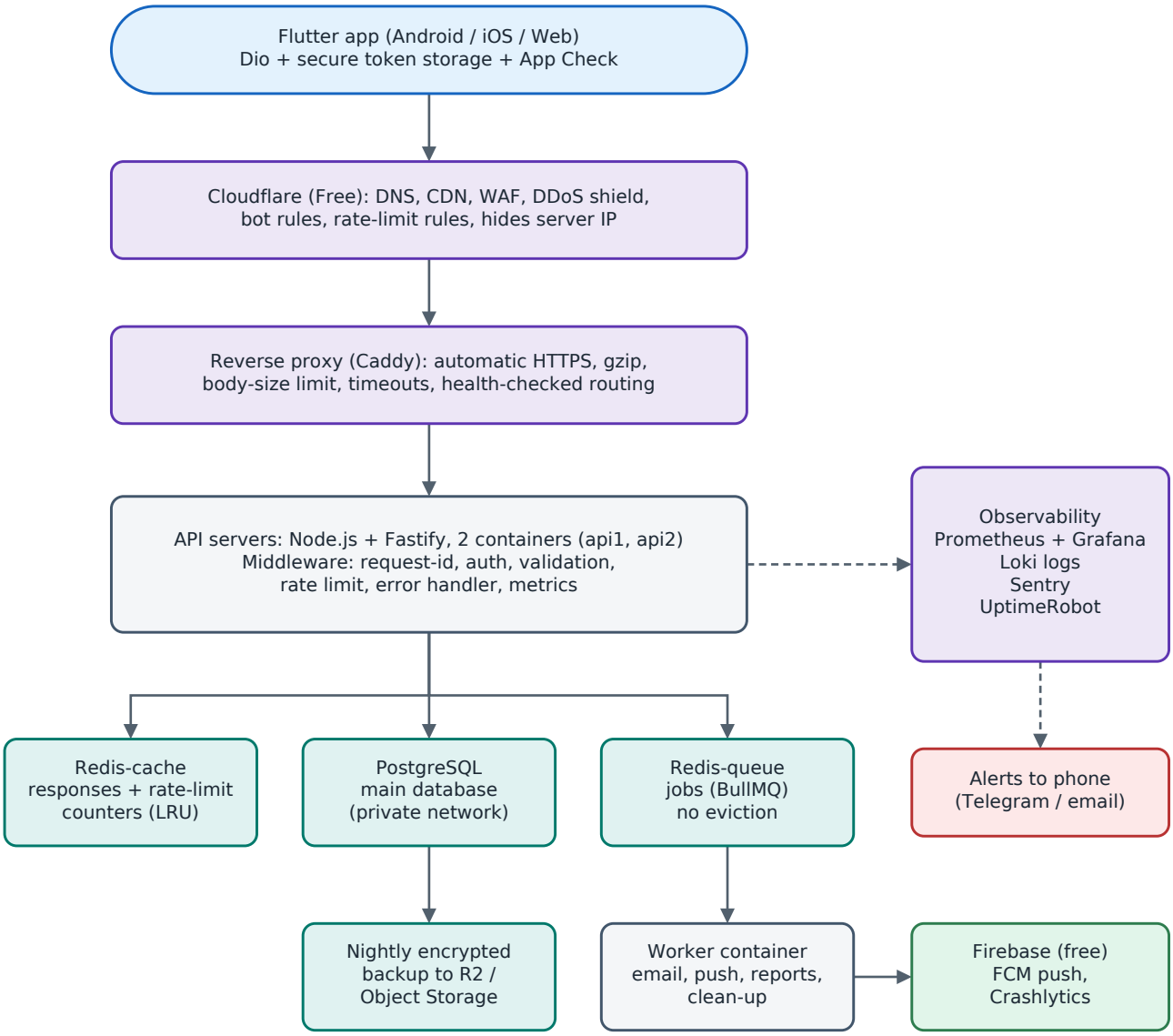



*Figure 1 — Full system architecture. Solid arrows = requests. Dashed arrows = monitoring and alerts.*

| Layer | Tool (free) | Its job | Protects against |
|---|---|---|---|
| **Flutter app** | Dio, flutter_secure_storage, Firebase App Check | Sends requests, keeps tokens safe, shows friendly errors | Fake clients, stolen tokens |
| **Cloudflare Free** | DNS, CDN, WAF, DDoS shield | Blocks attacks and bad bots before they reach your server. Caches public data. Hides your server IP. | DDoS, scanners, bot floods |
| **Reverse proxy** | Caddy | HTTPS certificates, compression, size limits, timeouts. Sends traffic only to healthy API containers. | Slow attacks, huge uploads, dead containers |
| **API servers** | Node.js + Fastify, 2 containers | Business logic, auth checks, validation, rate limits | Bad input, abuse, unauthorized access |
| **Redis-cache** | Redis with LRU eviction | Fast answers and rate-limit counters | Slow database, brute force |
| **PostgreSQL** | PostgreSQL 17 | Permanent data with rules (constraints, foreign keys) | Data corruption, lost records |
| **Redis-queue** | Redis + BullMQ, no eviction | Holds background jobs safely | Slow requests, lost jobs |

| Layer | Tool (free) | Its job | Protects against |
|---|---|---|---|
| **Worker** | Same code, different start command | Sends emails and push messages, builds reports, cleans old data | API slowing down from heavy tasks |
| **Backups** | pg_dump + age + rclone to R2 | Nightly encrypted copy stored away from the server | Server loss, human mistakes, ransomware |
| **Observability** | Prometheus, Grafana, Loki, Sentry, UptimeRobot | Shows problems early and sends alerts | Silent failures |

**Use two Redis containers, on purpose**
Cache data may be deleted when memory is full, and that is fine. Job-queue data must NEVER be deleted. So run one Redis for cache (`allkeys-lru`) and one Redis for the queue (`noeviction`). Mixing them is a classic mistake that silently loses jobs.

**Why two API containers?**
One container can restart, deploy, or crash while the other keeps answering. Caddy checks `/ready` every 10 seconds and sends traffic only to healthy containers. This is the cheapest form of high availability.

# 3. The free tool stack

Every tool below has a free option. Free-tier limits change often, so confirm each limit on the provider's own page before you rely on it (last checked November 2026).

| Need | Free choice | Why this one | Watch out for |
|---|---|---|---|
| **DNS, CDN, WAF, DDoS** | Cloudflare Free | Hides your server IP, blocks attacks, caches public GET data | Free plan has limited custom rules. Use “Full (strict)” SSL mode. |
| **Server** | Oracle Cloud Always Free VM (Ubuntu) | Always on. No sleeping. Free forever (if you stay in the limits). | Allowance was cut in 2026. Idle VMs may be reclaimed. Keep backups off this server. |
| **Containers** | Docker + Docker Compose | Same setup on laptop and server. Built-in restart policies and health checks. | Set log size limits or the disk fills up. |
| **HTTPS + proxy** | Caddy | Gets and renews HTTPS certificates automatically | Only open ports 80 and 443 to the world. |
| **API language** | Node.js (current LTS) + TypeScript + Fastify | Fast, huge ecosystem, easy JSON. Alternatives: NestJS, FastAPI (Python), Dart Frog or Serverpod (if you want Dart everywhere). | Pick one and stay with it. Do not mix frameworks. |
| **Database** | PostgreSQL 17 in Docker | Reliable, strict, free forever, great for money and user data | You must run backups. Free managed databases may pause or scale to zero when idle, which causes a slow first request. |
| **Cache + counters** | Redis (or Valkey) | Memory speed. Rate limits and sessions. | Set `maxmemory` or it can eat all RAM. |
| **Background jobs** | BullMQ (on Redis) | Retries, delays, schedules, dead-letter queue | Needs its own Redis with `noeviction`. |
| **Push notifications** | Firebase Cloud Messaging (FCM) | Free, standard for Flutter on Android and iOS | Remove dead tokens when FCM says they are invalid. |
| **Fake-app protection** | Firebase App Check | Checks that requests come from your real app | An extra layer only. Never your only protection. |
| **Crash reports** | Firebase Crashlytics (app) + Sentry (server) | See errors with stack traces before users complain | Remove personal data from reports. |
| **Monitoring** | Prometheus + Grafana + Loki, plus UptimeRobot from outside | Graphs, logs, and alerts. The outside check sees what users see. | Do not expose Grafana or /metrics to the internet without login. |
| **File storage** | Cloudflare R2 (free tier) or Oracle Object Storage | Store images and files. Use signed upload links. | Check the current free storage size and request limits. |
| **CI/CD** | GitHub Actions + Dependabot + CodeQL | Auto-test, auto-scan, auto-deploy | Private repos have a monthly free-minutes limit. |
| **Secrets** | GitHub Secrets + `.env` file on server (permission 600) | Keeps passwords out of code | Never commit `.env`. Rotate secrets after any leak. |
| **Email** | A free-tier email API (for example Brevo or Resend) | Verification and password-reset emails | Set up SPF, DKIM, DMARC or mail goes to spam. |
| **API docs** | OpenAPI + Swagger UI | Docs are generated from your schemas, so they never get old | Protect the docs page on production. |

## 3.1 RAM and disk budget for one free VM

Plan for the smaller (2 CPU, 12 GB RAM) allowance so you are safe even after the 2026 change. Set a memory limit on every container so one bad container cannot kill the others.

| Component | RAM limit | Note |
|---|---|---|
| PostgreSQL | 3 GB | Biggest user. Do not run it without a limit. |
| API api1 + api2 | 0.5 GB each (1 GB) | Node.js heap limit about 400 MB each |
| Worker | 0.5 GB | Runs background jobs |
| Redis-cache + Redis-queue | 0.3 GB + 0.2 GB | Cache uses `maxmemory 256mb` |

| Component | RAM limit | Note |
|---|---|---|
| Caddy | 0.1 GB | |
| Prometheus + Grafana + Loki | 1.5 GB | Short retention (7 to 14 days) to save disk |
| Operating system + Docker | about 1.5 GB | |
| **Free buffer** | **about 3.9 GB** | Keep 25-30% free for traffic spikes and the disk cache |

**Disk**
Free block storage is limited (Oracle documents 200 GB in total for free accounts, and the boot disk takes about 50 GB). Keep Docker logs small (`max-size 10m`, `max-file 3`), prune old Docker images weekly, and keep only 7 daily backups on the server (the rest live in R2).

# 4. The life of one request

This is the single most important flowchart. Every API call must walk through the same checks in the same order. The red boxes are the safe, clear answers the Flutter app receives when something is wrong.

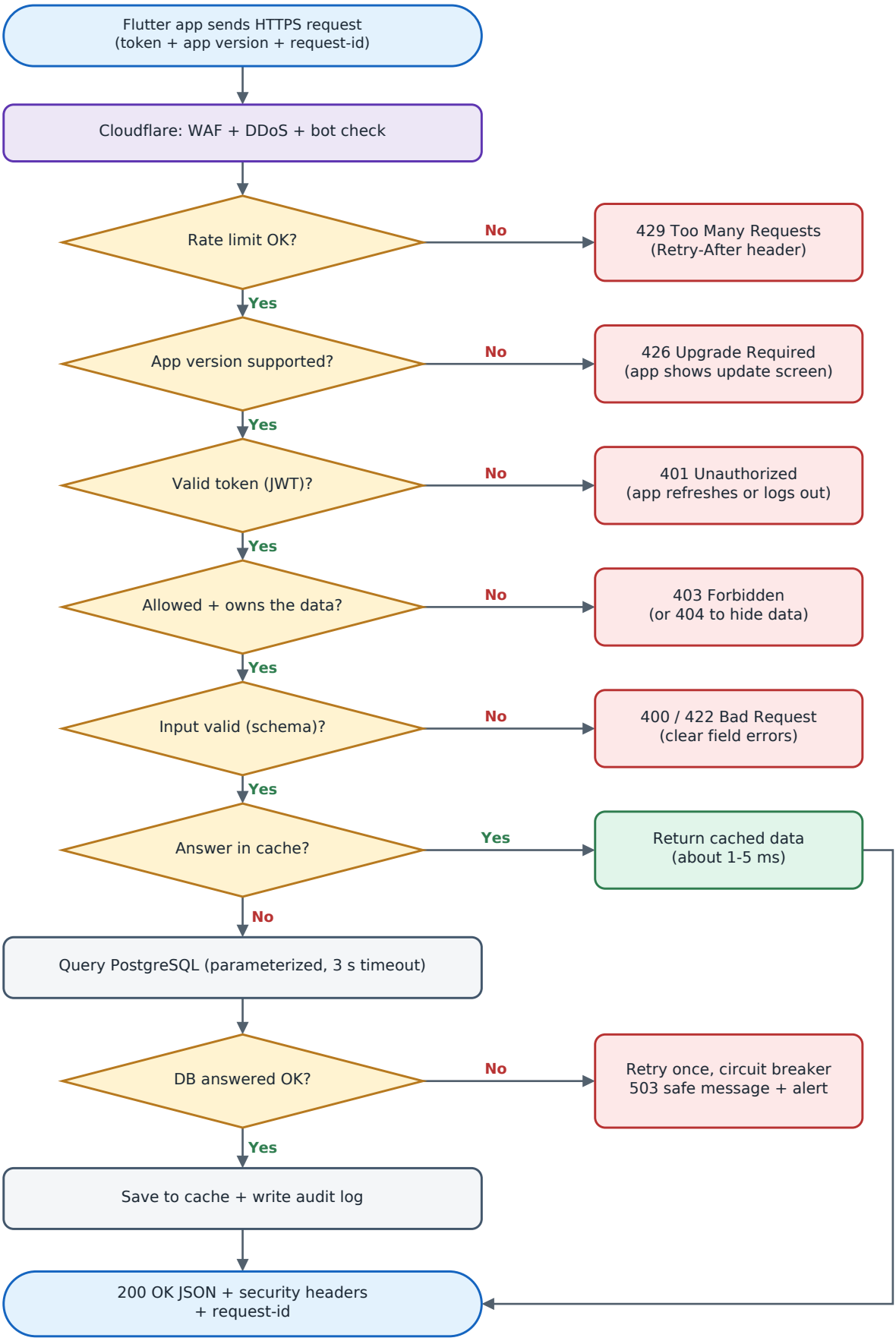



*Figure 2 — What happens to every API call, from the Flutter app to the JSON answer.*

| Step | What happens | Why it matters | If it fails |
|---|---|---|---|
| **1. Cloudflare** | WAF rules, DDoS shield, bot filter | Stops attacks early, before they cost you CPU | Challenge page or 403 |
| **2. Rate limit** | Counts requests per IP and per user in Redis | Stops brute force, scraping, and an app bug that loops forever | 429 + `Retry-After` |
| **3. App version** | Reads `X-App-Version` and compares with the minimum allowed | Lets you retire old or broken app versions safely | 426 (app shows update screen) |
| **4. Token check** | Verifies JWT signature, expiry, issuer, audience | Only logged-in users go further | 401 |
| **5. Permission + owner** | Checks the role AND that this record belongs to this user | Stops “Broken Object Level Authorization”, the #1 API risk | 403, or 404 to hide the record |
| **6. Validation** | A schema checks every field, type, and length | Blocks injection and bad data | 400 or 422 |
| **7. Cache** | Looks in Redis first | 1-5 ms answers and a relaxed database | (miss goes to step 8) |
| **8. Database** | Parameterized query with a 3-second timeout | Safe from SQL injection. Never hangs forever. | 503 (safe message) |
| **9. Save + log** | Writes to cache and the audit log | Faster next time. You can investigate later. | (logged, request still succeeds) |
| **10. Response** | JSON + security headers + request-id | The app can show the request-id to support | — |

> **Golden rule**
> Never build a “quick” endpoint that skips this chain. Put the chain in shared middleware, so a new endpoint gets all the protection automatically and a developer cannot forget it.

# 5. Every API, explained

The endpoints below are a complete starting set for a typical Flutter app (accounts, profile, your own data, files, notifications, admin). The “items” group is an example: rename it to your real data, for example orders, posts, tasks, or products. All paths start with `/api/v1`.

## 5.1 Rules that apply to every API

| Rule | Standard | Why |
|---|---|---|
| **Base URL + version** | `https://api.yourapp.com/api/v1/...` | Old app versions stay alive on `/v1` while you build `/v2`. Users do not update instantly. |
| **Format** | JSON, UTF-8. Dates in ISO 8601 UTC, for example `2026-11-08T10:30:00Z`. | One format means fewer bugs in Flutter. |
| **Names** | Plural nouns (`/items`), HTTP verbs for actions: GET read, POST create, PATCH change part, PUT replace, DELETE remove. | Predictable for every developer. |
| **Headers from the app** | `Authorization: Bearer <token>`, `X-App-Version: 1.4.2`, `X-Platform: android`, `X-Request-Id: <uuid>`, `Idempotency-Key: <uuid>` on create/pay calls. | Auth, force-update, tracing, and safe retries. |
| **Pagination** | Cursor style: `?limit=20&cursor=abc`. Max limit 100. Reply has `nextCursor`. | Fast on big tables. No “page 5000” slowdowns. |
| **Filtering and sorting** | Only allow-listed fields: `?status=open&sort=-createdAt` | Stops attackers sorting by secret columns or forcing slow queries. |
| **Safe retries** | POST that creates money or records needs an `Idempotency-Key`. The server stores the first answer for 24 h and returns it again if the app retries. | Mobile networks drop. Without this, users get double orders. |
| **Unknown fields** | Rejected (`additionalProperties: false`) | Stops “mass assignment” (a user sending `role: admin`). |
| **Size limits** | JSON body max 1 MB. Files go straight to R2 with a signed link, never through the API. | Protects memory and speed. |

### Standard success answer

```
HTTP 200
{
  "data": { "id": "0192f1c2-...", "title": "Buy milk", "done": false, "createdAt": "2026-11-08T10:30:00Z" },
  "meta": { "requestId": "3b1f6c0e-5d3a-4e0b-9a77-2f5f0c1d8a11" }
}
```

### Standard error answer (always the same shape)

```
HTTP 422
{
  "error": {
    "code": "VALIDATION_FAILED",
    "message": "Some fields are not valid.",
    "details": [ { "field": "email", "issue": "must be a valid email" } ],
    "requestId": "3b1f6c0e-5d3a-4e0b-9a77-2f5f0c1d8a11"
  }
}
```

The app shows `message` to the user and keeps `requestId` for support. The server never sends stack traces, SQL, or file paths in an error.

## 5.2 Group A: System and app configuration

| Method | Path | Who can call | What it does | What is checked |
|---|---|---|---|---|
| GET | `/health` | Public (local only) | Liveness: “is the process alive?” Returns `200` in under 5 ms. | No database call, so it never hangs. |
| GET | `/ready` | Local only | Readiness: can we reach PostgreSQL and Redis? Returns `503` if not. | Used by Caddy and Docker. Blocked from the internet at Caddy. |
| GET | `/metrics` | Local only | Prometheus numbers: requests, latency, errors, memory, queue size. | Never public. Blocked at Caddy and by firewall. |
| GET | `/api/v1/app/config` | Public | Minimum supported app version, maintenance mode flag, feature flags, store links. | Cached 60 s in Cloudflare. Lets you force an update or show a maintenance screen without a new release. |

## 5.3 Group B: Authentication

| Method | Path | Who can call | What it does | What is checked |
|---|---|---|---|---|
| POST | `/auth/register` | Public | Create an account with email + password. | Email format; password length 10+ and not in a common-password list; rate limit 10/hour per IP; email must be unique; sends a verification email. |
| POST | `/auth/verify-email` | Public (token) | Confirms the email with a one-time token from the email. | Token is random, single-use, expires in 24 h, stored as a hash. |
| POST | `/auth/login` | Public | Returns an access token and a refresh token (Figure 3). | Rate limit + lockout; Argon2id check; same generic error for wrong email or wrong password. |
| POST | `/auth/social` | Public | Login with Google or Apple. The app sends the provider's ID token. | Server verifies signature, `aud` (your client ID), `iss`, and expiry with the provider's public keys. Never trust the app's word. |
| POST | `/auth/refresh` | Refresh token | Swaps a refresh token for a new pair (Figure 4). | Token exists, not expired, not revoked; reuse detection revokes all sessions. |
| POST | `/auth/logout` | Logged in | Revokes this device's refresh token and removes its push token. | Idempotent: calling it twice is fine. |
| POST | `/auth/logout-all` | Logged in | Revokes every session of the user. | Needs a valid access token. |
| POST | `/auth/forgot-password` | Public | Sends a reset link or code by email. | Always answers `202` whether or not the email exists (no user guessing). Strict rate limit: 3/hour per email and IP. |
| POST | `/auth/reset-password` | Public (token) | Sets a new password with the emailed token. | Token single-use, expires in 30 min. Revokes all sessions after success. |
| POST | `/auth/change-password` | Logged in | Changes the password. | Needs the current password. Revokes other sessions. |

## 5.4 Group C: User profile and devices

| Method | Path | Who can call | What it does | What is checked |
|---|---|---|---|---|
| GET | `/users/me` | Logged in | Returns your own profile. | ID comes from the token, never from the URL, so users cannot read others. |
| PATCH | `/users/me` | Logged in | Changes name, language, avatar URL, and settings. | Allow-list of fields. `role`, `email_verified`, `id` can never be changed here. |
| DELETE | `/users/me` | Logged in + password | Deletes the account (soft delete, then permanent purge after 30 days). | Re-authentication required. Apple and Google Play require in-app account deletion for apps with sign-up. |
| POST | `/users/me/avatar-upload-url` | Logged in | Returns a short-lived signed URL so the app uploads the image straight to R2. | File type allow-list (jpeg, png, webp), size max 5 MB, link expires in 5 min, random file name. |
| POST | `/users/me/devices` | Logged in | Saves the phone's FCM push token. | Token format, max 10 devices per user, one row per token. |
| DELETE | `/users/me/devices/:id` | Logged in | Removes a push token (on logout). | Device must belong to the user. |
| GET | `/users/me/sessions` | Logged in | Lists active logins (device name, last used, city). | Only own sessions. |
| DELETE | `/users/me/sessions/:id` | Logged in | Logs out one device remotely. | Session must belong to the user. |

## 5.5 Group D: Your main data (example name: items)

| Method | Path | Who can call | What it does | What is checked |
|---|---|---|---|---|
| GET | `/items` | Logged in | List your items with cursor pagination, filters, and sorting. | Only rows where `user_id` = you. Allow-listed filters. `limit` max 100. Indexed query. |
| POST | `/items` | Logged in | Create an item. Returns `201` and the new item. | Schema validation, `Idempotency-Key`, per-user quota (for example 1000 items), no unknown fields. |
| GET | `/items/:id` | Logged in | Read one item. Cached in Redis (Figure 6). | Ownership check. Returns `404` (not `403`) for other users' items so IDs cannot be probed. |
| PATCH | `/items/:id` | Logged in | Change part of an item. | Ownership; `If-Match` version header (optimistic locking) so two phones do not overwrite each other silently. |
| DELETE | `/items/:id` | Logged in | Soft delete (sets `deleted_at`). Real purge happens later in a job. | Ownership. Lets users undo and protects against mistakes. |
| POST | `/uploads/sign` | Logged in | Signed URL for uploading a file to R2. | Content-type allow-list, max size, per-user daily limit, random key. |
| GET | `/home` | Logged in | One call returns everything the home screen needs (a “screen endpoint”). | Fewer round trips on slow mobile networks. Cached per user for 15-30 s. |

## 5.6 Group E: Jobs and notifications

| Method | Path | Who can call | What it does | What is checked |
|---|---|---|---|---|
| POST | `/reports/export` | Logged in | Starts a heavy job. Replies `202 Accepted` + `jobId` at once (Figure 7). | Per-user limit (for example 5/hour). Job saved before the answer is sent. |
| GET | `/jobs/:id` | Logged in | Shows job status: queued, running, done, failed + result link. | Job must belong to the user. |
| GET | `/notifications` | Logged in | In-app notification list (cursor pagination). | Own rows only. |
| POST | `/notifications/:id/read` | Logged in | Marks a notification as read. | Own rows only. Idempotent. |

## 5.7 Group F: Payments (only if your app sells something)

| Method | Path | Who can call | What it does | What is checked |
|---|---|---|---|---|
| POST | `/purchases/verify` | Logged in | The app sends the Google Play or App Store receipt. The server verifies it with the store and unlocks the feature. | Never trust the app saying “I paid”. Verify with the store from the server. Idempotent per transaction ID. |
| POST | `/webhooks/stripe` (or store notifications) | Signature only | Provider tells you a payment changed. | Verify the provider's signature header. Reject old timestamps. Store event IDs so the same event is never processed twice. |

## 5.8 Group G: Admin (separate role, extra protection)

| Method | Path | Who can call | What it does | What is checked |
|---|---|---|---|---|
| GET | `/admin/users` | Role `admin` | Search and list users. | Role check on the server for every call. Optional IP allow-list and 2FA (TOTP). Hide emails partly in the list. |
| PATCH | `/admin/users/:id/status` | Role `admin` | Ban or unban a user and revoke their sessions. | Every action written to `audit_logs` (who, what, when, from where). |
| GET | `/admin/audit-logs` | Role `admin` | Read the audit trail. | Read-only. Logs cannot be edited through the API. |
| POST | `/admin/broadcast` | Role `admin` | Send a push message to many users (as a queued job). | Rate limited. Preview first. Batched through the queue, never a loop inside a request. |

**Function-level access control**
A normal user must get `403` on every `/admin/*` path, even if they guess the URL. Test this for every admin endpoint. Put the role check in one shared hook for the whole `/admin` route group, not inside each handler.

## 5.9 What status code means, and what the Flutter app should do

| Code | Meaning | What the Flutter app does |
|---|---|---|
| **200 / 201 / 204** | OK / Created / Done with no body | Show the result. |
| **202** | Accepted: work continues in the background | Show “processing”, poll `/jobs/:id` or wait for the push. |
| **400 / 422** | Bad request / invalid fields | Show the field messages under the inputs. Do not retry. |
| **401** | Token missing or expired | Refresh the token once, then retry the call. If refresh fails, go to the login screen. |
| **403** | Logged in but not allowed | Show “no permission”. Do not retry. |
| **404** | Not found (or hidden) | Show an empty or “not found” state. |
| **409** | Conflict (version changed, duplicate) | Reload the data and ask the user to try again. |
| **413** | Body or file too large | Tell the user the size limit. Compress before upload. |
| **426** | App version too old | Show the “please update” screen with a store link. |
| **429** | Too many requests | Wait for `Retry-After` seconds, then retry. Slow the UI down. |
| **500** | Server bug (safe message) | Show “something went wrong” with the request-id. Retry once with a delay. |
| **502 / 503 / 504** | Server busy, restarting, or timed out | Retry with back-off (1 s, 2 s, 4 s, with a random jitter), max 3 tries, then show a retry button. |

# 6. Login, tokens, and sessions

Login is the most attacked part of any app. These two flowcharts show exactly how it is protected. Read them together with the rules table below.

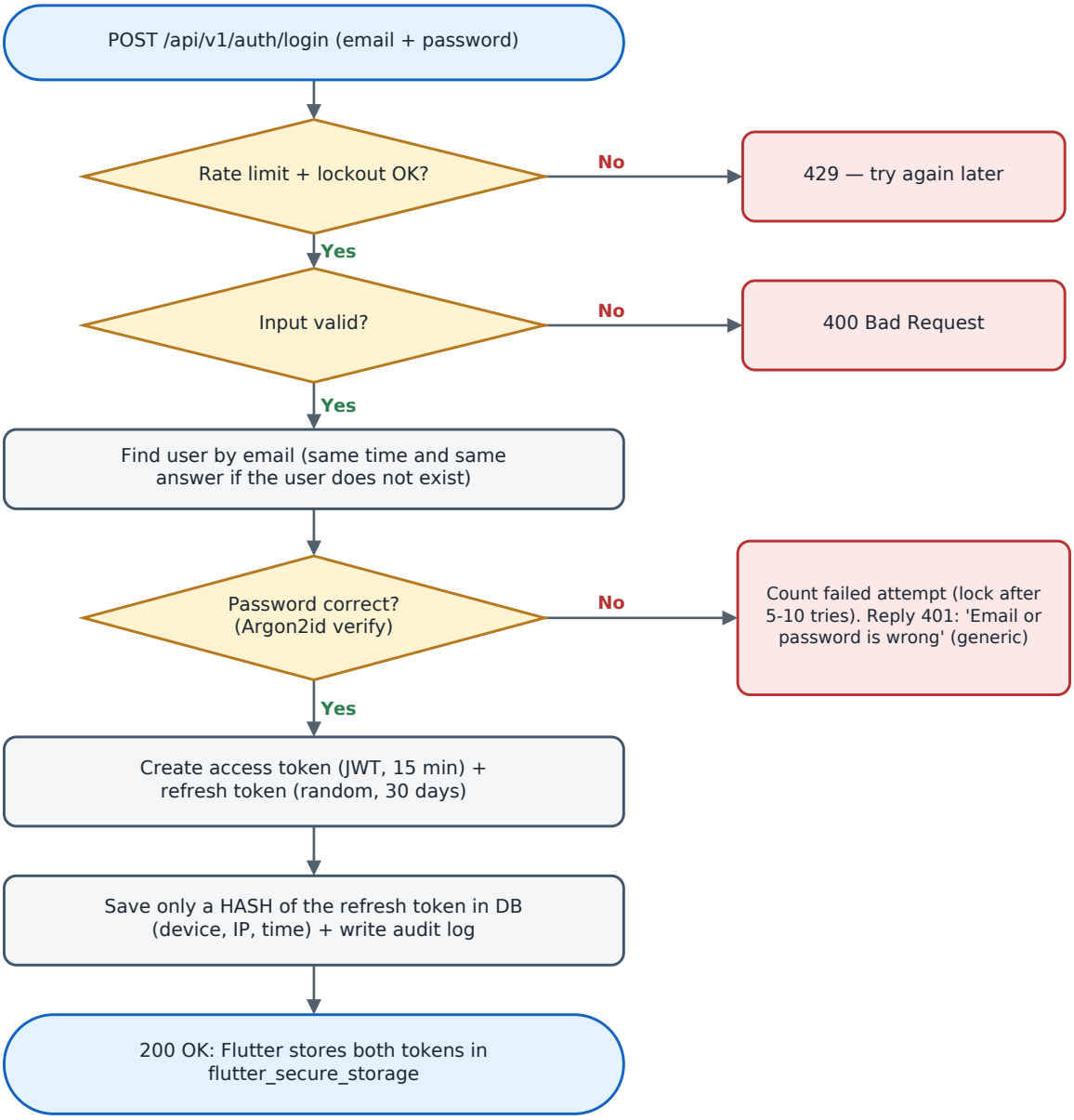



*Figure 3 — Login flow with rate limit, lockout, and hashed refresh-token storage.*

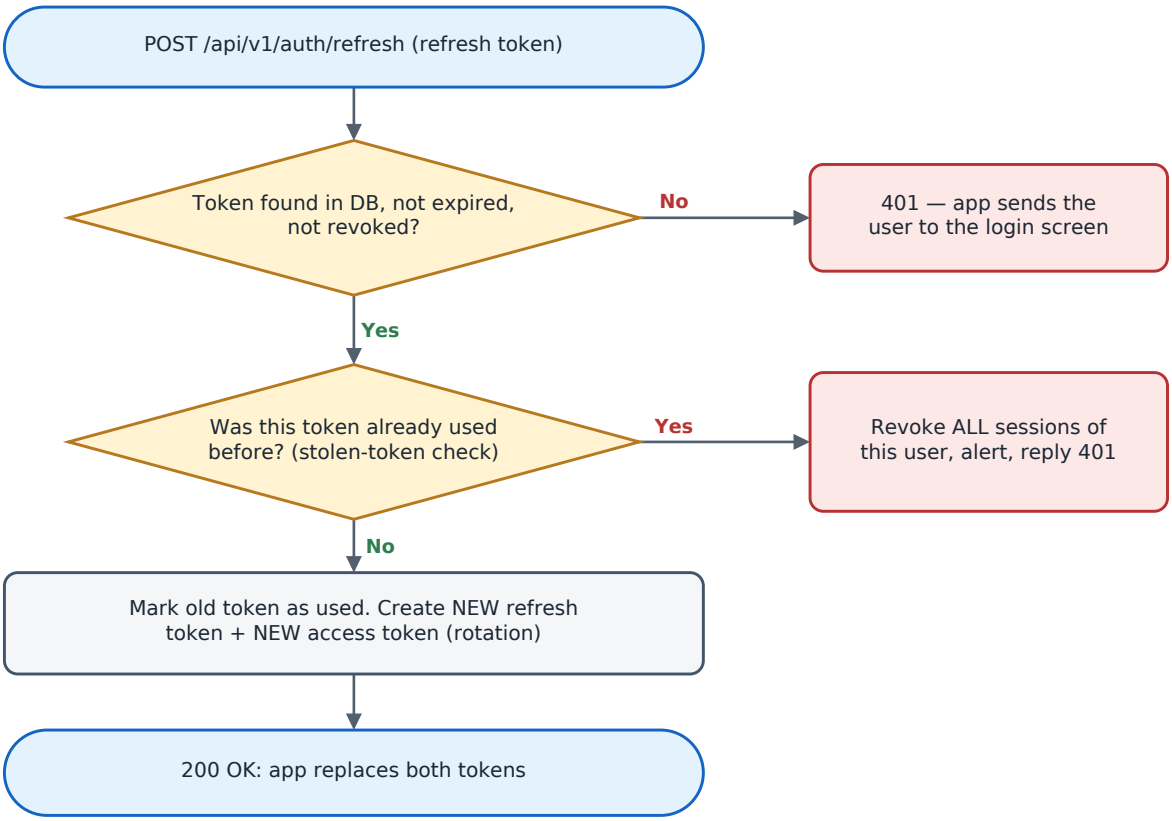



*Figure 4 — Token refresh with rotation and stolen-token detection.*

## 6.1 Token rules

| Item | Rule | Reason |
|---|---|---|
| **Access token** | JWT, lifetime 15 minutes. Contains `sub` (user id), `iss`, `aud`, `exp`, `iat`, `jti`. Small payload only. | If stolen, it dies quickly. |
| **Signing** | Use ES256 or EdDSA with a private key, or HS256 with a random secret of 32+ bytes. The server accepts ONLY the algorithm you chose (never `none`). | Blocks token-forgery tricks. |
| **Refresh token** | Random 256-bit value (not a JWT), lifetime 30 days, stored in the database as a SHA-256 hash with device, IP, and last-used time. | A database leak does not leak working tokens. |
| **Rotation** | Every refresh creates a new refresh token and marks the old one as used. If an old one comes back, revoke ALL sessions of that user. | Detects stolen tokens. |
| **Storage in Flutter** | `flutter_secure_storage` (Android Keystore, iOS Keychain). Never `SharedPreferences`, never plain files, never logs. | Other apps and backups cannot read them. |
| **Password hashing** | Argon2id (OWASP minimum: 19 MiB memory, 2 passes, 1 thread). Never MD5, SHA-1, or plain SHA-256. | Makes stolen hashes extremely expensive to crack. |
| **Password policy** | Minimum 10 characters, allow long passphrases, reject common and breached passwords. Do not force strange symbol rules. | Better real security and fewer forgotten passwords. |
| **Email verification** | Unverified accounts cannot do sensitive actions. Verification tokens are one-time. | Stops fake sign-ups. |
| **2FA** | Optional TOTP for users, required for admins. | A stolen password is not enough. |
| **Logout** | Revoke the refresh token on the server AND delete tokens in the app. | Real logout, not only a screen change. |

## 6.2 Rate limits (starting values)

| Endpoint | Limit | Key (what is counted) | On breach |
|---|---|---|---|
| `/auth/login` | 5 tries per 15 min | IP + email together | 429, then temporary lockout (15 min) |
| `/auth/register` | 10 per hour | IP | 429 |
| `/auth/forgot-password` | 3 per hour | email and IP | 202 (same answer), no email sent |
| **Public endpoints** | 30 per minute | IP | 429 |
| **Logged-in endpoints** | 100 per minute | user id | 429 + `Retry-After` |
| **Heavy endpoints (export, upload sign)** | 5 to 20 per hour | user id | 429 |

**Count by user AND by IP**
Many mobile users share one IP (mobile networks, offices). Counting only by IP blocks innocent people. Counting only by user lets attackers try unlimited accounts. Use both, and let Cloudflare handle raw floods.

# 7. Security: no weak spots

Good security is not one big lock. It is many small locks, so one mistake does not open the whole house. This is called **defense in depth**. Below is every layer, then the OWASP API Top 10 risks, then the Flutter app's part (because a safe server with an unsafe app is still unsafe).

## 7.1 The seven layers

| Layer | What we do |
|---|---|
| **1. Edge (Cloudflare)** | Managed WAF rules, DDoS shield, bot protection, rate-limit rules, hidden origin IP, Cloudflare Turnstile (free CAPTCHA) on sign-up and password reset. |
| **2. Network and server** | Firewall: only ports 80 and 443 are public, and only from Cloudflare IP ranges. SSH only with keys and only from your IP. PostgreSQL and Redis have NO public ports. fail2ban. Automatic security updates. |
| **3. Transport** | HTTPS everywhere, TLS 1.2 or higher, HSTS, Cloudflare SSL mode “Full (strict)”. No plain HTTP API at all. |
| **4. Application** | Authentication, authorization on every record, input validation, rate limits, safe error messages, security headers, audit logs. |
| **5. Data** | Least-privilege database users, encrypted backups, encrypted disks, minimal personal data, no secrets or tokens in logs. |
| **6. Supply chain** | Lock files, Dependabot, `npm audit`, CodeQL, Trivy image scans, pinned image versions, small base images. |
| **7. People and process** | 2FA on GitHub, Cloudflare, Oracle, and email accounts. Code review. Secret rotation. Tested backups. A written incident plan. |

## 7.2 OWASP API Security Top 10 (2023): risk by risk

| ID | Risk | What an attacker does | Our protection |
|---|---|---|---|
| **API1** | Broken object-level authorization | Changes `/items/41` to `/items/42` and reads someone else's data | Every query includes `user_id` from the token. Other users' records return 404. Random UUIDs instead of 1, 2, 3. Automated test with two users. |
| **API2** | Broken authentication | Guesses passwords, steals or forges tokens | Argon2id, rate limit, lockout, 15-minute JWT, rotating refresh token, fixed algorithm, generic login errors. |
| **API3** | Broken object-property authorization | Sends `{"role":"admin"}` or reads hidden fields | Input allow-list (unknown fields rejected). Response mappers return only safe fields. Never return the raw database row. |
| **API4** | Unrestricted resource consumption | Floods the API, asks for 1 million rows, uploads huge files | Rate limits, body limit 1 MB, page limit 100, timeouts, per-user quotas, queue for heavy work, container memory limits. |
| **API5** | Broken function-level authorization | A normal user calls `/admin/...` | One role check hook on the whole `/admin` group. Tests that expect 403. |
| **API6** | Unrestricted access to sensitive business flows | Bots create thousands of accounts, abuse coupons or OTP | Turnstile on sign-up, email verification, per-device and per-IP limits, alerts on strange spikes. |
| **API7** | Server-side request forgery (SSRF) | Gives a URL that makes your server call internal services | Do not fetch user-supplied URLs. If you must: domain allow-list, block private IP ranges, no redirects, short timeout. |
| **API8** | Security misconfiguration | Finds debug mode, open ports, default passwords, `CORS: *` | Hardened Compose file, config validated at startup, CORS allow-list, security headers, scans, no debug in production. |
| **API9** | Improper inventory management | Finds forgotten old or test APIs | OpenAPI file is the single source of truth. Sunset old versions on purpose. Staging is private. Delete unused endpoints. |
| **API10** | Unsafe consumption of other APIs | Poisons data coming from a third-party service | Validate third-party responses, set timeouts, verify webhook signatures, use TLS. |

## 7.3 Server hardening, step by step

| Item | How to do it |
|---|---|
| **SSH** | Keys only. Set `PasswordAuthentication no` and `PermitRootLogin no`. Restrict port 22 to your own IP in the cloud firewall. Run `fail2ban`. |
| **Firewall** | Cloud security list AND `ufw`: allow 80 and 443 (from Cloudflare ranges only), allow 22 (your IP only), deny the rest. Cloudflare publishes its IP list; refresh it monthly with a script. |
| **No public databases** | In Docker Compose, give `postgres`, `redis-cache`, and `redis-queue` NO `ports:` lines. They live on an internal Docker network only. |
| **Containers** | Run as a non-root user, `read_only: true`, `no-new-privileges`, drop all Linux capabilities you do not need, and set memory and CPU limits. |
| **Updates** | Enable `unattended-upgrades` for security patches. Rebuild Docker images weekly to pick up base-image fixes. |
| **Database users** | Three roles: `app` (read/write rows only), `migrator` (can change tables, used only during deploy), `backup` (read-only). The app user must not be able to `DROP` or `ALTER`. |
| **Time** | Keep the server clock correct (`chrony`). Wrong time breaks token expiry and logs. |
| **Disk encryption** | Cloud block volumes are normally encrypted at rest by default; confirm this in your console. Encrypt backups yourself with `age` before upload. |
| **Secrets** | A `.env` file with permission `600`, owned by the deploy user, outside the web root, never in Git. Different secrets for dev, staging, and production. |

## 7.4 Security headers and CORS

| Header or setting | Value | Why |
|---|---|---|
| **Strict-Transport-Security** | `max-age=31536000; includeSubDomains` | Browsers and clients always use HTTPS. |
| **X-Content-Type-Options** | `nosniff` | Stops content-type confusion. |
| **Referrer-Policy** | `no-referrer` | Does not leak URLs. |
| **Content-Security-Policy** | `default-src 'none'; frame-ancestors 'none'` (JSON API) | An API never needs to load scripts or be framed. |
| **Cache-Control** | `no-store` on login, profile, and token answers | Private data is never cached by a proxy. |
| **CORS** | Allow only your exact web origins (admin site, Flutter Web). Never `*` together with credentials. | Native Android and iOS apps do not use CORS. It only matters for browsers. |
| **Server header** | Remove it (`-Server` in Caddy) | Do not advertise the software version. |

## 7.5 The Flutter app's half of the security

Anything inside the app can be extracted by a determined attacker. So the app must be careful, and the server must never trust the app.

| Topic | Do this in the Flutter app | Why |
|---|---|---|
| **Token storage** | `flutter_secure_storage` only. Never `SharedPreferences`. | Keychain and Keystore protect the tokens. |
| **HTTPS only** | Block cleartext traffic on Android (`usesCleartextTraffic=false`). Keep iOS App Transport Security on. | No plain HTTP, ever. |
| **No secrets in the app** | No database passwords, no private keys, no server API keys. Only public IDs (Firebase config, Google client ID). | Everything in an app can be unpacked. |
| **Firebase App Check** | Turn it on (Play Integrity on Android, App Attest on iOS). The app sends the App Check token in a header. The server verifies it. | Blocks scripts that fake your app. |
| **Certificate pinning (optional)** | Pin the public key, include a backup pin, and rehearse certificate changes. If your team is small, you may skip pinning. | A wrong pin breaks every installed app until users update. Powerful but dangerous. |
| **Obfuscation** | `flutter build apk --obfuscate --split-debug-info=build/symbols`. Keep the symbol files for crash reports. | Slows reverse engineering. It is not a wall. |
| **Token refresh** | One shared refresh at a time (queue the other calls). Retry the failed call once. See the Dio code in Section 11. | Avoids many parallel refresh calls that trigger the “stolen token” alarm. |

| Topic | Do this in the Flutter app | Why |
|---|---|---|
| **Logging** | Never print tokens, passwords, or personal data. Only log in debug mode (`kDebugMode`). | Logs can be read from rooted phones and crash tools. |
| **Sensitive screens** | Optional: biometric lock (`local_auth`) and screenshot blocking. | Protects money or private data screens. |
| **Force update** | At app start, call `/app/config`. If the app version is below the minimum, show an update screen. | Retires broken or unsafe versions. |
| **Dependencies** | Run `flutter pub outdated` regularly. Use well-known packages. Remove unused ones. | Fewer supply-chain risks. |
| **Rooted / jailbroken phones** | Optional detection to show a warning or limit features. Never your only defense. | Easy to bypass, but raises the effort. |

## 7.6 Data protection and privacy

- Collect only what you need. Data you do not store cannot be stolen.
- Never store card numbers. Use the store billing or a payment provider (Stripe and similar) and keep only the provider's IDs.
- Hash refresh tokens and one-time tokens in the database. Mask emails and IDs in logs.
- Offer data export and account deletion (the app stores require deletion). Purge soft-deleted accounts after 30 days.
- Keep audit logs for about 12 months, then delete them. Keep application logs for 14 to 30 days.
- Publish a privacy policy, and follow the laws of the countries where your users live. This guide is not legal advice.

## 7.7 Free security testing tools

| Tool | What it finds | When to run it |
|---|---|---|
| `npm audit` + **Dependabot** | Known-vulnerable packages | Every pull request, plus weekly auto-updates |
| **gitleaks** | Passwords and keys committed by mistake | Every pull request (and as a pre-commit hook) |
| **CodeQL** | Code-level bugs such as injection | Every pull request |
| **Trivy** | Vulnerabilities in the Docker image and OS packages | Every build |
| **OWASP ZAP baseline scan** | Common web/API issues on a running staging server | Weekly against staging |
| **k6** | Speed under load, behavior when abused | Before every big release |
| **SSL test (SSL Labs)** | Weak HTTPS settings. Aim for grade A. | After setup and after changes |
| `nmap` **on your own server** | Open ports you did not expect (should be only 80 and 443) | After setup, then monthly |

### Manual tests to run before launch

1. Create two users. Log in as user A and try to read, change, and delete user B's records. Every call must fail (404 or 403).
2. Call every `/admin/*` endpoint as a normal user. Expect 403 each time.
3. Edit the access token (change one character, change the algorithm, remove the signature). Expect 401.
4. Send a 5 MB body, a wrong content-type, extra fields, very long strings, and SQL-like text (`' OR 1=1 --`). Expect 400, 413, or 422 and no server error.
5. Try 10 wrong passwords in a row. Expect a lockout and a generic message.
6. Re-use an old refresh token. Expect all sessions revoked.
7. Look at an error answer. It must not contain stack traces, SQL, file paths, or server names.

## 7.8 If something goes wrong: incident plan

1. **Detect.** An alert fires or a user reports it. Note the time.

2. **Contain.** Turn on Cloudflare “Under Attack” mode or block the IPs. Set maintenance mode in `/app/config`. Revoke all sessions: `UPDATE refresh_tokens SET revoked_at = now();`. Rotate the leaked secrets.
3. **Investigate.** Use audit logs, Loki logs, and Sentry. Find the entry point and the data touched.
4. **Fix and deploy.** Patch the hole, add a test that would have caught it, and deploy.
5. **Recover.** Restore from backup only if data was damaged.
6. **Tell people.** If personal data was exposed, tell the affected users and any regulator that your local law requires. Some laws set deadlines of a few days.
7. **Learn.** Write a short post-mortem: what happened, why, what we changed. Blame the process, not people.

# 8. No crash: reliability and self-healing

Crashes cannot be avoided completely, but they can be made rare, small, and short. The plan has three parts: **prevent** the common causes, **detect** problems in seconds, and **recover** by itself.

## 8.1 Why backends crash, and how we prevent each cause

| Common cause | Prevention in this design |
|---|---|
| **Unhandled error kills the process** | Global error handler. On a truly unknown error, log it and exit so Docker starts a clean process. A second container keeps serving meanwhile. |
| **Memory leak or huge response** | Container memory limit + Node heap limit, pagination, streaming for big data, alert at 90% memory. |
| **Database connections run out** | Pool size per container, `max_connections` budget, timeouts, PgBouncer if the app grows. |
| **One slow query blocks everything** | `statement_timeout`, indexes, slow-query log (200 ms), cache, no N+1 queries. |
| **Traffic spike or attack** | Cloudflare, rate limits, load shedding (fast 503), queue for heavy work. |
| **Disk full** | Docker log rotation, log retention, weekly prune, disk alert at 80%. |
| **Bad deploy** | CI tests, staging smoke test, rolling restart, auto-rollback (Figure 8). |
| **Third-party service is down** | Timeouts, retries with back-off, circuit breaker, queue so the user action still succeeds. |
| **HTTPS certificate expires** | Caddy renews automatically. Alert if less than 14 days remain. |
| **Wrong database change** | Backward-compatible migrations, tested on staging with a copy of data, backups before every deploy. |
| **Server lost or deleted** | Nightly backups off the server and a rebuild script (target: back online in 60 minutes). |

## 8.2 The reliability rules

| Rule | What to do | Starting value |
|---|---|---|
| **1. Timeouts everywhere** | Never wait forever for anything. | Outgoing HTTP 5 s. DB query `statement_timeout` 3 s. Redis 200 ms. Incoming request 15 s. |
| **2. Retry safely** | Retry only safe (idempotent) calls. Use exponential back-off plus random jitter. | 3 tries: about 200 ms, 400 ms, 800 ms. |
| **3. Circuit breaker** | After many failures, stop calling the broken service for a short time and use a fallback. | Open after 5 failures in 30 s. Try again after 30 s. |
| **4. Pool limits** | Total connections from all containers must stay below the database limit. | Pool 10 per container x 3 processes = 30, below `max_connections` 50. |
| **5. Memory limits** | Docker limit AND Node heap limit, so Node cleans up before Docker kills it. | Docker 512 MB, `--max-old-space-size=384`. |
| **6. Graceful shutdown** | On SIGTERM: stop new requests, finish running ones, close pools, exit. | Hard stop after 10 s. |
| **7. Global error handling** | One error handler for the whole API. Log with request-id. Send a safe message. | Exit on `unhandledRejection` so Docker restarts clean. |
| **8. Health checks** | `/health` (alive, no dependencies) and `/ready` (DB + Redis ok). | Every 10-15 s. |
| **9. Two containers** | api1 and api2 behind Caddy. `restart: unless-stopped` on everything. | Deploy one at a time. |
| **10. Load shedding** | When the event loop is too slow, answer 503 fast instead of piling up work. | Event-loop delay above 1 s. |
| **11. Queue slow work** | Email, push, reports, imports, and clean-up run in the worker. | API answers `202` in under 100 ms. |
| **12. Safe migrations** | Add first, switch code, remove later. Use `CREATE INDEX CONCURRENTLY`. Never rename and drop in the same release. | Every migration reviewed. |
| **13. Logs and disk** | Rotate logs. Alert on disk space. | 10 MB x 3 files per container. Alert at 80%. |
| **14. Pinned versions** | Lock files and `npm ci`. Docker image tags with exact versions, not `latest`. | Dependabot opens update pull requests. |
| **15. Kill switches** | A feature flag or maintenance flag in `/app/config` lets you switch off one feature without a release. | Cached 60 s. |
| **16. Practice** | Once a month stop a container on purpose and restore a backup on a test machine. | Written in the runbook. |

## 8.3 Self-healing flow

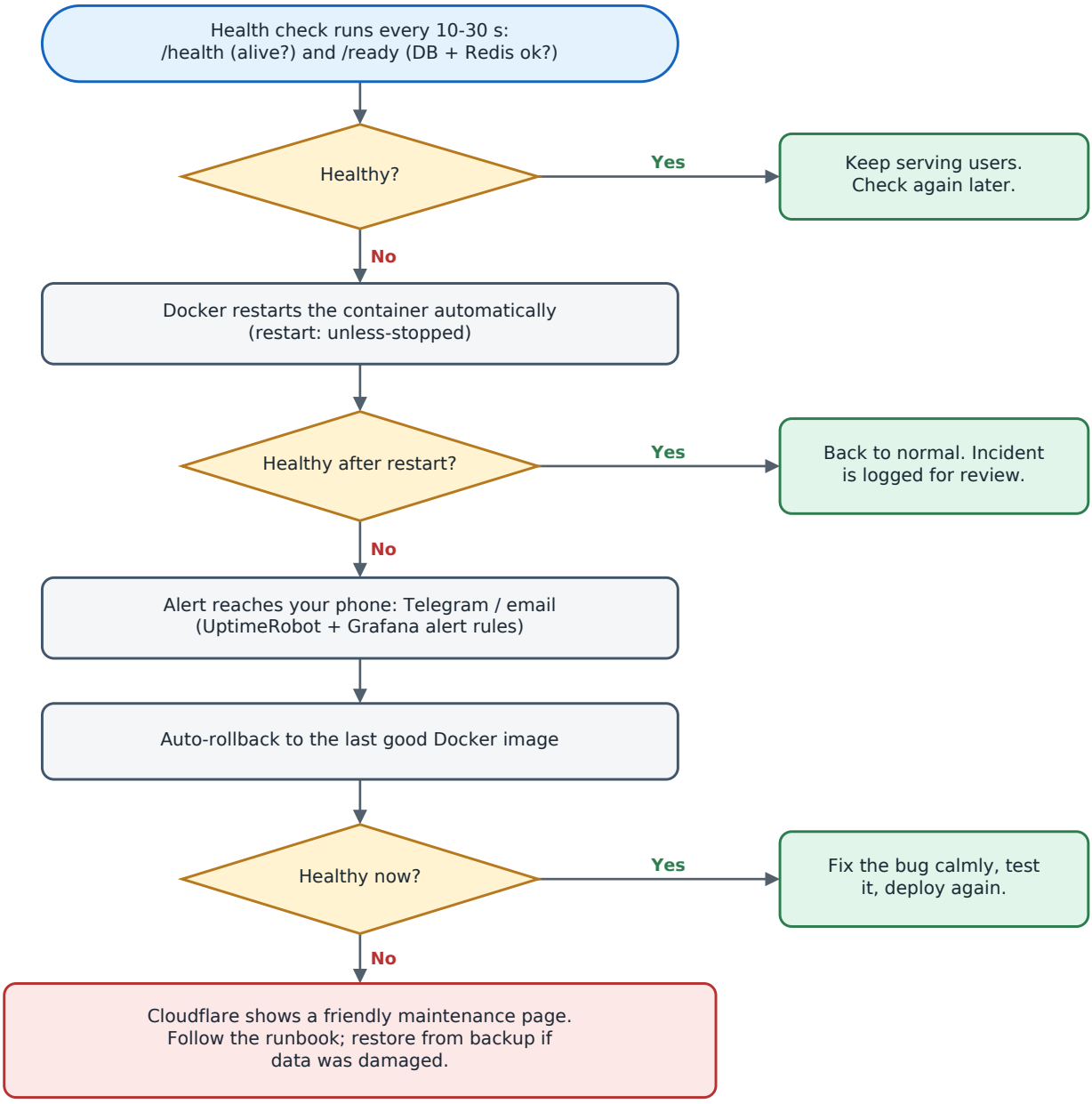



*Figure 5 — What happens automatically when a container becomes unhealthy.*

Three tools make this work together. **Docker** restarts a stopped or unhealthy container. **Caddy** stops sending users to a container that fails `/ready`, so users do not notice a single failing container. **UptimeRobot** (outside your server) alerts you when the whole site is unreachable, which is the case your own server can never report.

## 8.4 Backups and disaster recovery

A backup you have never restored is only a hope. Plan two numbers. **RPO** is how much data you can afford to lose (daily dumps: up to 24 hours; with WAL archiving: a few minutes). **RTO** is how long you can be offline (aim for 60 minutes with a written rebuild).

| What | How | How often | Kept for | Where |
|---|---|---|---|---|
| **PostgreSQL full backup** | `pg_dump -Fc`, encrypted with age | Daily, 02:15 | 7 daily + 4 weekly + 3 monthly | 7 days on the VM, all copies in R2 |
| **WAL archive (upgrade)** | WAL-G or pgBackRest | Continuous | 7 days | R2. Gives point-in-time recovery (RPO of minutes). |
| **Code and config** | Compose file, Caddyfile, scripts, dashboards as JSON | Every commit | Forever | Private GitHub repo |
| **Secrets** | Encrypted copy of `.env` | When changed | Latest + previous | Password manager (not the server) |

| What | How | How often | Kept for | Where |
|---|---|---|---|---|
| **Uploaded files** | R2 bucket with versioning or a weekly copy to a second bucket | Continuous / weekly | 30 days of versions | R2 |

### Restore drill (do this every month on a test machine)

1. Create a fresh Ubuntu machine or container. Install Docker.
2. Clone the private repo and put the `.env` file in place.
3. Run `docker compose up -d postgres`.
4. Download the newest backup from R2 and decrypt it with `age -d`.
5. Restore with `pg_restore -d appdb --clean --if-exists`, then run a few test queries.
6. Write down how long it took and what surprised you. Fix the runbook.

> **Backup trap**
> Never keep the only backup on the same server. If that server is deleted, reclaimed, or hacked, the backup disappears with it. Always copy backups to a different provider or account.

# 9. No delay: speed rules

Users feel delay when a screen waits more than about one second. Your server cannot control the phone's network, but it can control everything else. The goal: **95% of API calls finish in under 300 ms on the server side**, and cached answers in under 20 ms.

## 9.1 Where the time goes

| Step | Rough time | What we do about it |
|---|---|---|
| **Phone network (mobile data, Wi-Fi)** | 50-300 ms | Not in your control. Reduce the number of calls and the size of answers. |
| **Cloudflare edge + TLS** | 5-30 ms | HTTP/2 or HTTP/3 and connection reuse. Cache public answers at the edge. |
| **Caddy proxy** | under 1 ms | Nothing to do. |
| **API code** | under 20 ms | No blocking code. No heavy work inside requests. |
| **Redis** | 1-5 ms | Cache hot data with a TTL. |
| **PostgreSQL query** | under 30 ms | Indexes, no N+1 queries, pagination, short timeouts. |
| **JSON + compression** | under 5 ms | Small answers, gzip/zstd. |

## 9.2 Techniques, from biggest win to smallest

| Technique | How | Effect |
|---|---|---|
| **Always-on server** | No sleeping free hosts. Keep two containers warm. | Removes 30-60 s cold starts. |
| **Redis cache-aside** | Read cache first. On a miss, read the DB and store with a TTL (Figure 6). | From about 20 ms to 2 ms per read. |
| **Database indexes** | Index every column used in `WHERE`, `JOIN`, and `ORDER BY`. Use composite indexes such as `(user_id, created_at DESC, id)`. Check plans with `EXPLAIN (ANALYZE, BUFFERS)`. | From seconds to milliseconds on big tables. |
| **Fix N+1 queries** | Use one JOIN or one `WHERE id = ANY($1)` instead of one query per row. | Often 10-50x faster lists. |
| **Cursor pagination** | Use `WHERE (created_at, id) < ($1, $2) LIMIT 20` instead of `OFFSET 100000`. | Stays fast on page 5000. |
| **Screen endpoints** | `GET /home` returns everything one screen needs in one call. | Fewer round trips on slow mobile networks. |
| **Small answers** | Return only needed fields. Compress with gzip or zstd. Use `ETag` and `304 Not Modified`. | Less data over the air. |
| **Queue heavy work** | Reports, emails, image processing, imports run in the worker (Figure 7). | API stays under 100 ms. |
| **Signed direct uploads** | The app uploads files straight to R2 with a signed link. | API memory and CPU stay free. |
| **Connection reuse** | Keep-alive to the database (pool), to Redis, and for outgoing HTTP. | Saves handshakes. |
| **Edge caching** | Cache public GET answers in Cloudflare (`Cache-Control: public, max-age=60`). Never cache private data. | Many requests never reach your server. |
| **Don't block Node.js** | No synchronous CPU work, no giant `JSON.parse`. Argon2 runs in a thread pool, and its concurrency is limited. | The event loop stays free. |
| **Server near users** | Pick the cloud region closest to most of your users. | Saves 50-200 ms per call. |

## 9.3 Cache-aside, step by step

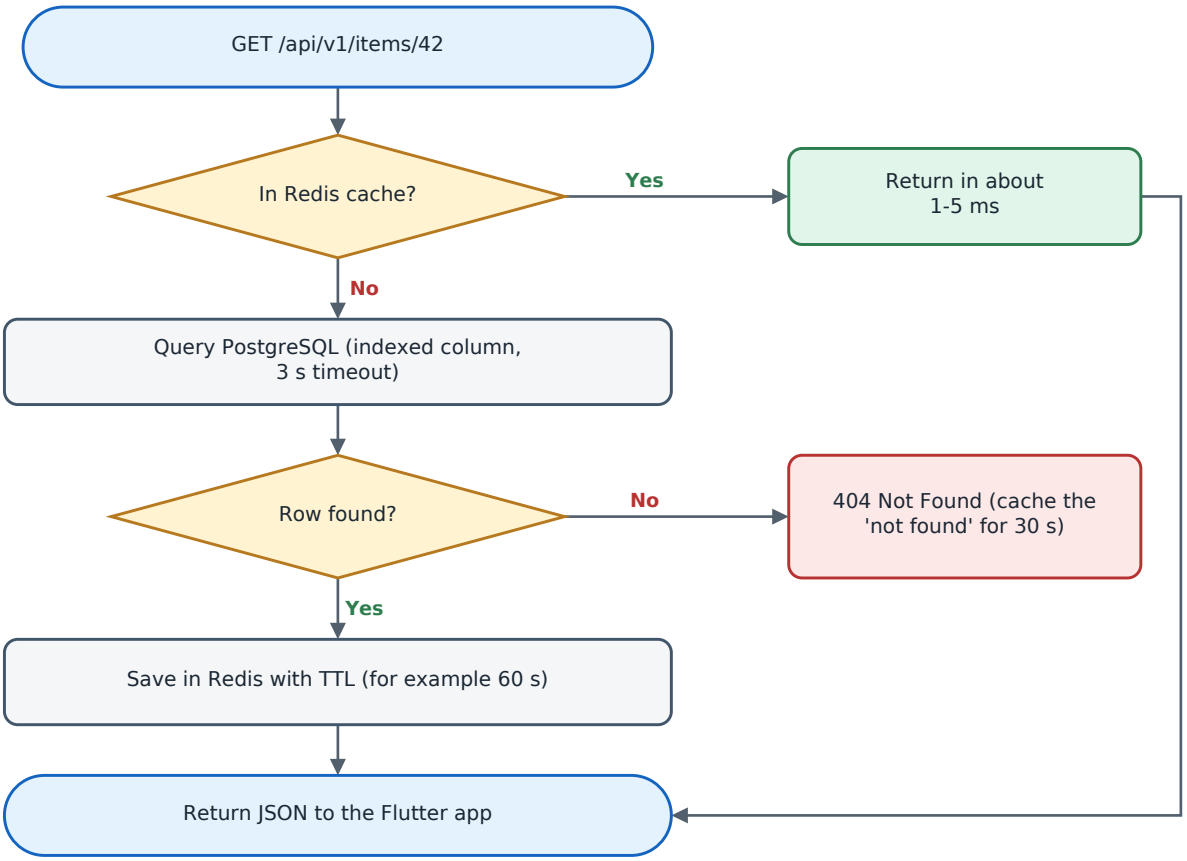



*Figure 6 — Reading one record with Redis cache-aside.*

| Cache rule | Details |
|---|---|
| **Key names** | `v1:items:{userId}:{itemId}`. The user id is in the key, so one user can never receive another user's cached data. |
| **TTL** | 30-300 s for most data. Add 10% random jitter so many keys do not expire at the same second. |
| **On writes** | After a successful DB write, delete the cache key (or write the new value). Do it after the transaction commits. |
| **Hot keys** | For a very popular key, take a short Redis lock so only one request rebuilds it (prevents a “stampede”). |
| **Negative cache** | Cache “not found” for 30 s so attackers cannot hammer the database with missing IDs. |
| **If Redis is down** | Skip the cache and read the database (a little slower, still correct). Never fail the whole request. |
| **Private data** | Send `Cache-Control: private, no-store` so Cloudflare never stores personal answers. |

## 9.4 Heavy work without waiting

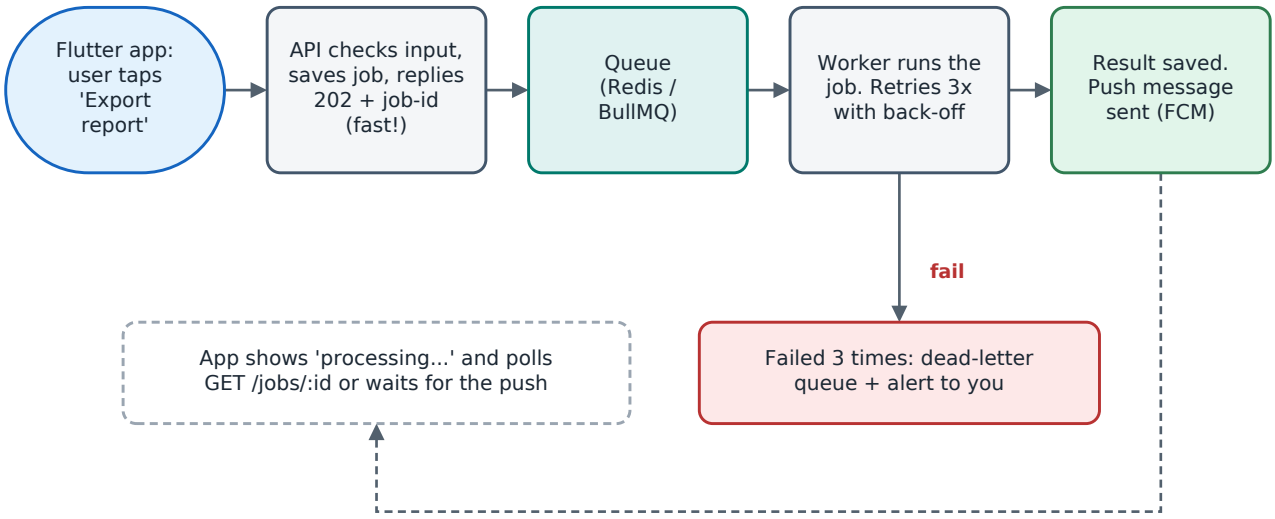



*Figure 7 — A slow task becomes a job. The user gets an immediate answer.*

Rule of thumb: if a request could take more than about 500 ms, or calls another company's server, or sends something, or touches thousands of rows, it belongs in the queue. The API saves the job, answers `202`, and the worker finishes it. The worker retries 3 times with growing waits. A job that still fails goes to a dead-letter list and sends you an alert, so nothing disappears silently.

## 9.5 Flutter-side speed tips

| Tip | How |
|---|---|
| **Show cached data first** | Keep the last answer in a local database (for example `drift` or `sqflite`) and show it at once. Refresh in the background. |
| **Paginate everything** | Infinite scroll with the `nextCursor` value. Load 20 items at a time. |
| **Cancel old requests** | Use Dio `CancelToken` when the user leaves a screen or types a new search. |
| **Debounce search** | Wait 300-400 ms after the last key before calling the API. |
| **Retry smartly** | On network errors and 502/503/504: back-off with jitter, max 3 tries. Never retry 4xx errors. |
| **Images** | Use `cached_network_image`, request the right size, and compress before upload. |
| **Optimistic UI** | Update the screen at once and fix it if the server says no. |
| **Fewer calls** | Use the `/home` screen endpoint. Do not call 8 small APIs when one will do. |
| **Offline** | Detect no connection, queue user actions locally, and send them with an `Idempotency-Key` later. |

## 9.6 Load test before launch

Use the free tool **k6** (script in Section 11). Test at 3 times your expected peak for 5 minutes. You pass when all of these are true:

- Error rate under 1%.
- 95th percentile (p95) under 300 ms for normal reads and under 800 ms for writes.
- CPU under 70% and memory not growing over time (no leak).
- Database connections under 80% of the pool, no slow queries above 200 ms.
- After the test, the system returns to normal speed by itself.

# 10. Easy to manage

A system you cannot understand at 3 a.m. is a risk. These rules keep the system small, tidy, and boring: one repository, one Compose file, one deploy command, one dashboard.

## 10.1 Project layout

```
backend/
├─ docker-compose.yml          # the whole system in one file
├─ Caddyfile                   # HTTPS, routing, headers
├─ .env.example                # every setting, with fake values (the real .env is never in Git)
├─ Makefile                    # make up | logs | deploy | backup | restore | status
├─ scripts/                    # backup.sh  deploy.sh  restore.sh
├─ monitoring/                 # prometheus.yml, Grafana dashboards (JSON), Loki config, alert rules
└─ api/
   ├─ Dockerfile
   ├─ openapi.yaml             # the API contract (generated from schemas)
   ├─ migrations/              # numbered SQL files: 001_users.sql, 002_items.sql ...
   └─ src/
      ├─ server.ts             # starts the app, graceful shutdown
      ├─ worker.ts             # starts the background worker
      ├─ config.ts             # reads and VALIDATES environment variables (crash early)
      ├─ plugins/              # security, rate-limit, auth, errors, metrics
      ├─ modules/
      │  ├─ auth/  users/  items/  admin/     # each: routes.ts  service.ts  repo.ts  schema.ts
      ├─ jobs/                 # queue definitions and job handlers
      └─ lib/                  # db.ts  redis.ts  mailer.ts  logger.ts
```

## 10.2 Three layers inside each module

| Layer | Does | Must NOT |
|---|---|---|
| **routes.ts** | Reads the HTTP request, runs the schema check, calls the service, sends the answer. | Contain business rules or SQL. |
| **service.ts** | Business rules: who may do what, limits, steps, calls to cache and queue. | Know about HTTP or SQL text. |
| **repo.ts** | All SQL for this module, always with parameters (`$1`, `$2`). | Contain business decisions. |

This makes each file small and easy to test. When a bug appears, you know which file to open.

## 10.3 One command for everything

| Command | What it does |
|---|---|
| `make up` | Starts the whole system (`docker compose up -d`). |
| `make status` | Shows container health, disk, memory, and the last backup time. |
| `make logs` | Shows live logs for the API (`docker compose logs -f api1 api2`). |
| `make migrate` | Runs new database migrations. |
| `make deploy` | Runs `scripts/deploy.sh`: build, migrate, rolling restart, health check, auto-rollback. |
| `make backup` | Runs a backup now. |
| `make restore FILE=...` | Restores a backup (asks for confirmation). |

## 10.4 Environments

| Environment | Where | Data | Rules |
|---|---|---|---|
| **Development** | Your laptop with Docker Compose | Fake data | Same containers as production. Debug logs allowed. |
| **Staging** | Same server, other port or hostname, protected by a password or Cloudflare Access | Fake or scrubbed data. Never real user data. | Every release goes here first. Smoke tests run here. |
| **Production** | The real server | Real data | Only code that passed CI and staging. No manual edits on the server. |

**Config rule:** every setting comes from environment variables and is validated at startup (for example with `zod`). If a variable is missing or wrong, the app refuses to start. It is much better to fail during deploy than at 3 a.m.

## 10.5 Automatic release pipeline

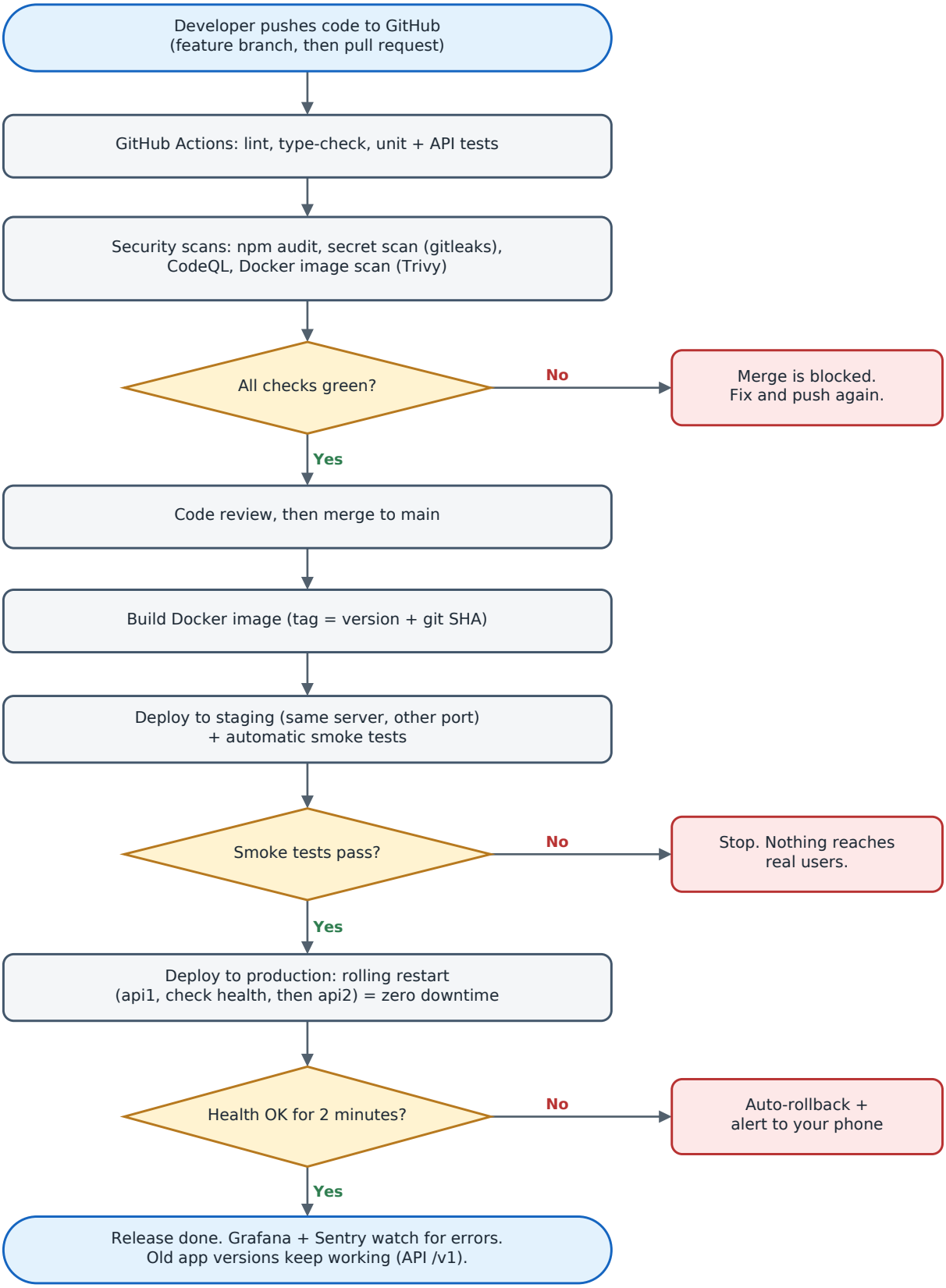



*Figure 8 — CI/CD: from `git push` to a safe, health-checked release.*

Nobody deploys by typing commands on the server. A person merges a pull request, and the pipeline does the rest. If any check fails, the release stops. If the health check fails after release, the system rolls back by itself.

## 10.6 Logging standard

Write one JSON object per line. The same fields everywhere make searching in Grafana Loki simple.

```
{"time":"2026-11-08T10:30:00.123Z","level":"info","requestId":"3b1f6c0e-...","userId":"0192f1c2-...",
 "method":"GET","route":"/api/v1/items/:id","status":200,"ms":14,"appVersion":"1.4.2","platform":"android"}
```

- Levels: `error` (needs action), `warn` (strange but handled), `info` (normal requests), `debug` (off in production).
- Never log passwords, tokens, full emails, card data, or request bodies. Mask them.
- Log the route pattern (`/items/:id`), not the raw URL, so metrics group nicely.
- Keep application logs 14 to 30 days, audit logs about 12 months.

## 10.7 Versioning without breaking old apps

- A Flutter app in a user's pocket may be 6 months old. The API must keep working for it.
- Only make **additive** changes inside `/v1`: new optional fields and new endpoints. Never rename, remove, or change the meaning of a field.
- For a breaking change, create `/v2`, keep `/v1` alive, and send a `Sunset` header on `/v1` with the end date.
- Use `/app/config` to set the minimum app version. When the old version is below 2% of users, retire it with a `426` answer.
- The Flutter app should ignore unknown JSON fields, so new server fields never crash old apps.

## 10.8 Documentation people really use

- **README**: how to start the system on a laptop in 5 commands.
- **openapi.yaml + Swagger UI** (staging only): every endpoint, always up to date because it is generated from the schemas.
- **Runbook** (Section 13): symptom, check, fix, in plain steps.
- **Decision notes**: one short paragraph per big choice (why Fastify, why two Redis). Future you will say thanks.
- **Rebuild guide**: how to create the whole system from an empty server in under 60 minutes.

# 11. Ready-to-use code and config

These samples are starting points, not magic. Replace `yourapp.com`, `myapp`, and the names with your own, and test every file on staging first.

## 11.1 docker-compose.yml

Two API containers, one worker, PostgreSQL, two Redis containers, and Caddy. Databases have no public ports. Every container has a memory limit, a restart policy, and log rotation.

```
# docker-compose.yml  -  start everything with:  docker compose up -d
x-api: &api
  image: myapp-api:latest
  build: ./api
  restart: unless-stopped
  env_file: .env
  networks: [internal]
  mem_limit: 512m
  cpus: 0.75
  read_only: true
  tmpfs: [/tmp]
  cap_drop: [ALL]
  security_opt: ["no-new-privileges:true"]
  depends_on:
    postgres:    { condition: service_healthy }
    redis-cache: { condition: service_healthy }
    redis-queue: { condition: service_healthy }
  healthcheck:
    test: ["CMD", "node", "dist/healthcheck.js"]    # tiny script: calls /health, exit 0 or 1
    interval: 15s
    timeout: 3s
    retries: 3
    start_period: 20s
  logging: { driver: json-file, options: { max-size: "10m", max-file: "3" } }

services:
  caddy:
    image: caddy:2
    restart: unless-stopped
    ports: ["80:80", "443:443"]              # the ONLY published ports
    volumes: ["./Caddyfile:/etc/caddy/Caddyfile:ro", "caddy_data:/data"]
    networks: [internal]
    mem_limit: 128m
    depends_on: [api1, api2]

  api1: *api
  api2: *api

  worker:
    <<: *api
    command: ["node", "dist/worker.js"]
    healthcheck: { disable: true }

  postgres:
    image: postgres:17
    restart: unless-stopped
    env_file: .env                           # POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB
    volumes: ["pg_data:/var/lib/postgresql/data"]
    networks: [internal]
    mem_limit: 3g
    shm_size: 256m
    command: ["postgres", "-c", "max_connections=50", "-c", "shared_buffers=768MB",
              "-c", "log_min_duration_statement=200"]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U $${POSTGRES_USER} -d $${POSTGRES_DB}"]
      interval: 10s
      retries: 5

  redis-cache:                               # may delete old keys when full: that is fine
    image: redis:7-alpine
    restart: unless-stopped
    command: ["redis-server", "--maxmemory", "256mb", "--maxmemory-policy", "allkeys-lru",
              "--save", "", "--requirepass", "${REDIS_PASSWORD}"]
    environment: { REDIS_PASSWORD: "${REDIS_PASSWORD}" }
    networks: [internal]
    mem_limit: 300m
    healthcheck:
      test: ["CMD-SHELL", "redis-cli -a $$REDIS_PASSWORD ping | grep -q PONG"]
      interval: 10s
      retries: 5
```

```
  redis-queue:                              # job data must NEVER be evicted
    image: redis:7-alpine
    restart: unless-stopped
    command: ["redis-server", "--maxmemory-policy", "noeviction", "--appendonly", "yes",
              "--requirepass", "${REDIS_PASSWORD}"]
    environment: { REDIS_PASSWORD: "${REDIS_PASSWORD}" }
    volumes: ["queue_data:/data"]
    networks: [internal]
    mem_limit: 256m
    healthcheck:
      test: ["CMD-SHELL", "redis-cli -a $$REDIS_PASSWORD ping | grep -q PONG"]
      interval: 10s
      retries: 5

networks: { internal: {} }
volumes: { pg_data: {}, caddy_data: {}, queue_data: {} }
```

## 11.2 Dockerfile for the API

```
# api/Dockerfile  (in production pin an exact Node version, for example node:24.x-alpine)
FROM node:lts-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build && npm prune --omit=dev

FROM node:lts-alpine
WORKDIR /app
ENV NODE_ENV=production
COPY --from=build /app/node_modules ./node_modules
COPY --from=build /app/dist ./dist
COPY --from=build /app/package.json ./
USER node                                   # never run as root
EXPOSE 3000
CMD ["node", "--max-old-space-size=384", "dist/server.js"]
```

## 11.3 Caddyfile

Cloudflare SSL mode should be “Full (strict)”. The health and metrics paths are hidden from the internet. For an outside uptime check, use a public path such as `/api/v1/app/config`.

```
api.yourapp.com {
    encode zstd gzip

    request_body {
        max_size 2MB
    }

    @private path /metrics /ready /health
    respond @private 404                    # never public

    reverse_proxy api1:3000 api2:3000 {
        lb_policy round_robin
        health_uri /ready                   # Caddy asks each container if it is ready
        health_interval 10s
        health_timeout 3s
        fail_duration 30s                   # stop using a failed container for 30 s
        transport http {
            dial_timeout 3s
            response_header_timeout 15s
        }
    }

    header {
        Strict-Transport-Security "max-age=31536000; includeSubDomains"
        X-Content-Type-Options "nosniff"
        Referrer-Policy "no-referrer"
        -Server
    }
}
```

## 11.4 Fastify server: security, health, errors, graceful shutdown

```
// src/server.ts  (ESM + TypeScript)
import Fastify from 'fastify';
import helmet from '@fastify/helmet';
import rateLimit from '@fastify/rate-limit';
import underPressure from '@fastify/under-pressure';
import { randomUUID } from 'node:crypto';
import { db, redis } from './lib/clients.js';        // your pg Pool and Redis client

const app = Fastify({
  logger: { level: 'info', redact: ['req.headers.authorization', 'req.headers.cookie'] },
  genReqId: (req) => (req.headers['x-request-id'] as string) || randomUUID(),
  bodyLimit: 1_000_000,          // 1 MB
  requestTimeout: 15_000,        // 15 s
  trustProxy: true,              // we sit behind Cloudflare + Caddy
});

await app.register(helmet);
await app.register(rateLimit, {
  max: 100, timeWindow: '1 minute', redis,
  keyGenerator: (req) => (req as any).user?.id ?? req.ip,   // user id when logged in, else IP
});
await app.register(underPressure, {                  // load shedding: fast 503 instead of a pile-up
  maxEventLoopDelay: 1000, maxHeapUsedBytes: 400 * 1024 * 1024, retryAfter: 5,
});

app.get('/health', async () => ({ status: 'ok' }));  // alive? (no dependencies)
app.get('/ready', async (_req, reply) => {           // can we really work?
  try { await db.query('select 1'); await redis.ping(); return { status: 'ready' }; }
  catch { return reply.code(503).send({ status: 'not-ready' }); }
});

// ... register modules here: auth, users, items, admin ...

app.setErrorHandler((err, req, reply) => {           // one place for all errors
  req.log.error({ err }, 'request failed');
  const status = err.statusCode && err.statusCode < 500 ? err.statusCode : 500;
  reply.code(status).send({ error: {
    code: status === 500 ? 'INTERNAL' : (err.code ?? 'BAD_REQUEST'),
    message: status === 500 ? 'Something went wrong.' : err.message,   // no stack traces, no SQL
    requestId: req.id,
  } });
});

for (const sig of ['SIGTERM', 'SIGINT'] as const) {  // graceful shutdown
  process.on(sig, async () => {
    app.log.info(`${sig}: draining`);
    setTimeout(() => process.exit(1), 10_000).unref(); // hard stop after 10 s
    await app.close();                                 // finish running requests
    await db.end(); await redis.quit();
    process.exit(0);
  });
}
process.on('unhandledRejection', (e) => { app.log.fatal(e); process.exit(1); }); // Docker restarts us

await app.listen({ host: '0.0.0.0', port: 3000 });
```

## 11.5 Backup script (cron: 15 2 * * *)

```
#!/usr/bin/env bash
# /opt/app/scripts/backup.sh
set -euo pipefail
cd /opt/app
set -a; source .env; set +a            # loads POSTGRES_USER, POSTGRES_DB, AGE_PUBLIC_KEY, HEALTHCHECK_PING_URL
STAMP=$(date -u +%Y%m%d-%H%M)
FILE=/opt/app/backups/app-$STAMP.dump.age
mkdir -p /opt/app/backups

docker compose exec -T postgres pg_dump -U "$POSTGRES_USER" -Fc "$POSTGRES_DB" \
  | age -r "$AGE_PUBLIC_KEY" > "$FILE"            # encrypted: only your private key can open it
test -s "$FILE"                                   # fail if the file is empty

rclone copy "$FILE" r2:app-backups/daily/         # copy OFF the server (rclone remote named "r2")
find /opt/app/backups -name '*.age' -mtime +7 -delete        # keep 7 days on the VM
curl -fsS -m 10 "$HEALTHCHECK_PING_URL" > /dev/null          # tell the monitor: backup succeeded
```

## 11.6 Deploy script with automatic rollback

```
#!/usr/bin/env bash
# /opt/app/scripts/deploy.sh  -  zero-downtime rolling deploy
set -euo pipefail
cd /opt/app
PREV=$(git rev-parse HEAD)
git pull --ff-only
docker compose build api1                                  # builds image myapp-api:latest
docker compose run --rm api1 npm run migrate               # migrations must be backward compatible

for svc in api1 api2; do                                   # one container at a time
  docker compose up -d --no-deps "$svc"
  for i in $(seq 1 30); do                                 # wait up to 60 s for "healthy"
    state=$(docker inspect -f '{{.State.Health.Status}}' "$(docker compose ps -q "$svc")")
    [ "$state" = healthy ] && continue 2
    sleep 2
  done
  echo "Health check failed for $svc - rolling back"
  git checkout "$PREV"
  docker compose build api1 && docker compose up -d --no-deps "$svc"
  exit 1
done
docker compose up -d --no-deps worker
echo "Deploy OK"
```

## 11.7 GitHub Actions: test, scan, deploy

gitleaks-action is free for personal accounts; organizations may need a license, in which case run the gitleaks program directly. Add a Trivy step to scan the Docker image.

```
# .github/workflows/ci.yml
name: CI
on:
  pull_request:
  push:
    branches: [main]
jobs:
  check:
    runs-on: ubuntu-latest
    defaults: { run: { working-directory: api } }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: lts/*, cache: npm, cache-dependency-path: api/package-lock.json }
      - run: npm ci
      - run: npm run lint && npm run typecheck && npm test
      - run: npm audit --audit-level=high
      - uses: gitleaks/gitleaks-action@v2
        env: { GITHUB_TOKEN: "${{ secrets.GITHUB_TOKEN }}" }

  deploy:
    needs: check
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - name: Deploy over SSH
        run: |
          echo "${{ secrets.DEPLOY_KEY }}" > key && chmod 600 key
          echo "${{ secrets.HOST_KEY }}" > known_hosts       # the server's SSH host key, saved as a secret
          ssh -i key -o UserKnownHostsFile=known_hosts deploy@${{ secrets.HOST }} /opt/app/scripts/deploy.sh
```

## 11.8 k6 load test

```
// loadtest.js  -  run:  k6 run loadtest.js
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  stages: [
    { duration: '1m', target: 50 },     // ramp up to 50 virtual users
    { duration: '3m', target: 50 },     // stay there
    { duration: '1m', target: 0 },      // ramp down
  ],
  thresholds: {
    http_req_failed:   ['rate<0.01'],   // under 1% errors
    http_req_duration: ['p(95)<300'],   // 95% under 300 ms
  },
};

export default function () {
  const r = http.get('https://staging-api.yourapp.com/api/v1/app/config');
  check(r, { 'status is 200': (x) => x.status === 200 });
  sleep(1);
}
```

## 11.9 Flutter: Dio interceptor with safe token refresh

`QueuedInterceptor` handles calls one at a time, so ten parallel 401 errors cause only one refresh call. This is simplified: add your own navigation to the login screen and handle refresh failures with care.

```
// lib/network/auth_interceptor.dart
import 'package:dio/dio.dart';
import 'package:flutter_secure_storage/flutter_secure_storage.dart';

class AuthInterceptor extends QueuedInterceptor {
  AuthInterceptor(this.dio, this.storage, this.baseUrl, this.appVersion);
  final Dio dio;
  final FlutterSecureStorage storage;
  final String baseUrl, appVersion;

  @override
  Future<void> onRequest(RequestOptions o, RequestInterceptorHandler h) async {
    final token = await storage.read(key: 'access');
    if (token != null) o.headers['Authorization'] = 'Bearer $token';
    o.headers['X-App-Version'] = appVersion;
    h.next(o);
  }

  @override
  Future<void> onError(DioException e, ErrorInterceptorHandler h) async {
    final retried = e.requestOptions.extra['retried'] == true;
    if (e.response?.statusCode == 401 && !retried) {
      try {
        final refresh = await storage.read(key: 'refresh');
        final res = await Dio(BaseOptions(baseUrl: baseUrl))     // plain Dio: no interceptor loop
            .post('/auth/refresh', data: {'refreshToken': refresh});
        await storage.write(key: 'access', value: res.data['data']['accessToken']);
        await storage.write(key: 'refresh', value: res.data['data']['refreshToken']);
        final req = e.requestOptions..extra['retried'] = true;
        req.headers['Authorization'] = 'Bearer ${res.data['data']['accessToken']}';
        return h.resolve(await dio.fetch(req));                  // repeat the original call once
      } catch (_) {
        await storage.deleteAll();                               // refresh failed: log out
        // TODO: navigate to the login screen here
      }
    }
    if (e.response?.statusCode == 426) {
      // TODO: show the "please update the app" screen
    }
    h.next(e);
  }
}
```

# 12. Database design that stays fast and safe

## 12.1 Rules for every table

- Primary key: UUID v7 (time-ordered, good for indexes) or a big integer identity. Do not expose sequential numbers in the API.
- Always add `created_at` and `updated_at` (`timestamptz`, stored in UTC). Add `deleted_at` where users can delete things (soft delete).
- Every table that belongs to a user has a `user_id` column with an index and a foreign key.
- Use constraints for rules: `NOT NULL`, `UNIQUE`, `CHECK`, foreign keys. The database is the last line of defense against bad data.
- Add a `version` integer for optimistic locking on records that two phones may edit.
- Change the schema only with numbered migration files, kept in Git. Never edit production tables by hand.
- Use parameters (`$1`, `$2`) in every query. Never build SQL by joining strings.

## 12.2 Core tables

| Table | Important columns | Notes |
|---|---|---|
| `users` | id, email (unique, lowercase), password_hash, email_verified_at, role, status, created_at, updated_at, deleted_at | Role is `user` or `admin`. Status can be `active` or `banned`. |
| `auth_identities` | id, user_id, provider (`google` / `apple`), provider_user_id | Unique on (provider, provider_user_id). |
| `refresh_tokens` | id, user_id, token_hash (unique), device_name, ip, user_agent, created_at, last_used_at, expires_at, used_at, revoked_at, replaced_by | Never store the token itself, only its hash. Purge expired rows daily. |
| `devices` | id, user_id, fcm_token (unique), platform, app_version, last_seen_at | Delete tokens that FCM reports as invalid. |
| `items` **(example)** | id, user_id, title, ..., version, created_at, updated_at, deleted_at | Index `(user_id, created_at DESC, id)` for the list screen. |
| `idempotency_keys` | user_id, key, request_hash, status_code, response_body, created_at | Unique on (user_id, key). Delete after 24 hours. |
| `audit_logs` | id, actor_user_id, action, target_type, target_id, ip, meta (jsonb), created_at | Append-only. The app user can INSERT but not UPDATE or DELETE. |
| `notifications` | id, user_id, type, payload (jsonb), read_at, created_at | Index `(user_id, created_at DESC)`. |

## 12.3 PostgreSQL starting settings (for about 3 GB of RAM)

| Setting | Starting value | Why |
|---|---|---|
| `max_connections` | 50 | Fewer, reused connections are faster. Use a pool in the API. |
| `shared_buffers` | 768MB (about 25%) | Main memory cache of PostgreSQL. |
| `effective_cache_size` | 2GB | Helps the planner choose good plans. |
| `work_mem` | 8MB | Per sort or hash step. Bigger numbers multiply with parallel work. |
| `log_min_duration_statement` | 200 (ms) | Logs every slow query so you can fix it. |
| **Role** `app`**:** `statement_timeout` | 3s | `ALTER ROLE app SET statement_timeout = '3s';` A runaway query is cancelled. |
| **Role** `app`**:** `idle_in_transaction_session_timeout` | 10s | Stops forgotten transactions from blocking others. |
| `pg_stat_statements` | On | Shows the slowest and most frequent queries. |
| **Autovacuum** | On (default) | Never turn it off. It prevents table bloat. |

These are starting points. Measure with real traffic and change one setting at a time.

# 13. Monitoring, alerts, and runbooks

You cannot fix what you cannot see. Watch the **four golden signals** and let the system call you, so you do not need to stare at screens.

| Signal | What to measure | Where |
|---|---|---|
| **Latency** | p50, p95, p99 response time per route | Prometheus (from `/metrics`) + Grafana |
| **Traffic** | Requests per second, active users, sign-ups, logins | Prometheus + Grafana |
| **Errors** | 5xx rate, 4xx rate (especially 401, 429), failed jobs, app crashes | Prometheus, Sentry, Crashlytics |
| **Saturation** | CPU, memory, disk, DB connections, event-loop delay, queue backlog | Node exporter, postgres exporter, redis exporter |

## 13.1 Alerts (send to Telegram or email)

| Alert | Condition | Level | First action |
|---|---|---|---|
| **Site down** | UptimeRobot check fails 2 times in a row (outside check) | Critical | Open the runbook “API down”. |
| **Error rate high** | 5xx above 2% for 5 minutes | Critical | Check Sentry and the last deploy. Roll back if it started after a release. |
| **Slow** | p95 above 800 ms for 10 minutes | Warning | Check slow-query log, cache hit rate, CPU. |
| **CPU** | Above 85% for 10 minutes | Warning | Find the busy container. Look for a loop or a traffic spike. |
| **Memory** | Container above 90% of its limit | Warning | Look for a leak. Restart the container if needed. |
| **Disk** | Above 80% used | Warning | Prune Docker images, check log size, check backups folder. |
| **DB connections** | Above 80% of the pool or `max_connections` | Warning | Look for long transactions or leaked connections. |
| **Queue** | More than 1000 waiting, or oldest job older than 5 minutes | Warning | Check the worker logs. Scale the worker count. |
| **Failed jobs** | Anything in the dead-letter queue | Warning | Read the error and fix or replay the job. |
| **Backup** | No successful backup ping for 26 hours | Critical | Run `make backup` and read the script output. |
| **Certificate** | Expires in under 14 days | Warning | Check Caddy logs and DNS. |
| **Restarts** | A container restarts more than 3 times in 10 minutes | Critical | Read the logs of that container. Roll back. |
| **Security** | More than 100 failed logins per minute, or a spike of 401/403 | Warning | Check Cloudflare analytics. Enable stricter rules. |

## 13.2 Dashboards to build

- **Overview:** requests per second, error percent, p50/p95/p99, uptime, current app versions in use.
- **Infrastructure:** CPU, memory, disk, network, container restarts.
- **Database:** connections, slow queries, cache-hit ratio, table and index size.
- **Redis and queue:** memory, hit rate, evictions, jobs waiting, active, failed, oldest job age.
- **Product:** sign-ups, logins, and the main actions of your app, so you see when something silently breaks.

## 13.3 Runbook: symptom, check, fix

| Symptom | Check first | Likely fix |
|---|---|---|
| **API down** | `make status`. Are containers healthy? `docker compose logs --tail 100 api1`. | `docker compose up -d`. If a release caused it, run the rollback. If the VM is gone, follow the rebuild guide. |
| **Everything slow** | Grafana: CPU, event-loop delay, DB slow queries, Redis memory. | Add the missing index, raise the cache TTL, restart the busy container, block abusive IPs. |
| **Many 401 errors** | Did the signing key change? Is the server clock correct? Did a release change token rules? | Fix the key or time. Check the app's refresh logic. |
| **Many 429 errors** | Cloudflare and API logs: one IP or many users? | Block the abuser. If real users are hurt (shared IPs), raise the per-IP limit. |
| **Database full or slow** | Disk space, table sizes, long transactions, locks. | Delete old rows (expired tokens, old audit logs), `VACUUM`, add disk, kill the long query. |
| **Disk full** | `df -h` and `docker system df`. | `docker system prune`, shorten log retention, move backups to R2. |
| **Push messages not arriving** | Worker logs, FCM error codes, device token age. | Delete invalid tokens. Check Firebase credentials. Check the queue. |
| **Login fails for everyone** | Argon2 CPU load, Redis rate-limit keys, database health. | Restart the API. Check that Redis is up. Roll back a recent auth change. |
| **Suspected attack** | Cloudflare analytics, 401/403 spikes, audit logs. | Turn on “Under Attack” mode. Follow the incident plan (Section 7.8). |

# 14. When free is not enough: the upgrade path

The free design above is honest and solid for a new or small app, but it has one big weakness: **one server**. If that single VM disappears, you are down until you rebuild it. The cheapest big improvements cost a few dollars a month, so plan them early.

| Stage | Typical situation | What to change | Rough cost |
|---|---|---|---|
| **0. Launch** | Under a few thousand users | Everything on one free VM, as in this guide. Off-server backups. Tested rebuild script. | Free |
| **1. First real users** | You care about every minute of downtime | Add a second small server (paid) as a warm standby, or move the API to a paid always-on host. Keep Cloudflare in front. | About $5-10 per month |
| **2. Growing** | Database is the busiest part | Move PostgreSQL to its own server or a managed database with automatic backups. Add WAL archiving (point-in-time restore). | Starts around $10-25 per month |
| **3. Busy** | Worker or API needs more power | Separate worker server. Add read replica. Use PgBouncer. Raise cache hit rate before buying hardware. | Depends on traffic |
| **4. Large** | High traffic or strict uptime promises | Several API servers behind a load balancer, managed Redis, multi-zone database, more careful on-call process. | Budget needed |

**Make the rebuild easy now**
Keep everything needed to recreate the system in Git: Compose file, Caddyfile, scripts, monitoring config, and a `cloud-init` or setup script that installs Docker and starts the stack. Then a lost server is a 1-hour job instead of a disaster. It also makes moving to a bigger server simple.

**Prices change**
Provider prices and free allowances change often. Treat the money amounts in this section as rough guides and check current price pages before you decide.

# 15. Master checklist: check every single thing

Print this page or copy it into your project tracker. Do not launch until every box is ticked, and ask a second person to verify the security part.

## A. Security

| | Check | How to verify |
|---|---|---|
| ☐ | All traffic is HTTPS, HTTP redirects to HTTPS, HSTS is on | SSL Labs test shows grade A |
| ☐ | Cloudflare proxy, WAF managed rules, and bot protection are on | Cloudflare dashboard |
| ☐ | Server accepts web traffic only from Cloudflare; port 22 only from your IP | `nmap` from outside shows only 80/443 |
| ☐ | PostgreSQL and Redis have no public ports | `nmap` plus `docker compose config` shows no `ports:` for them |
| ☐ | SSH: keys only, no root login, fail2ban running | `sshd -T`, `fail2ban-client status` |
| ☐ | Automatic security updates are on | `unattended-upgrades --dry-run` |
| ☐ | No secrets in Git or in the Flutter app | gitleaks passes; app package searched for keys |
| ☐ | `.env` has permission 600 and differs per environment | `ls -l .env` |
| ☐ | Passwords hashed with Argon2id; refresh tokens stored only as hashes | Read the code; look at the database |
| ☐ | Access token lives 15 minutes; refresh rotation and reuse detection work | Manual test (Section 7.7) |
| ☐ | Login, register, and forgot-password are rate limited and locked after repeated failures | 10 wrong passwords test |
| ☐ | Every query is filtered by the logged-in user (no BOLA) | Two-user test on every endpoint |
| ☐ | Every `/admin/*` route gives 403 to normal users | Automated test |
| ☐ | All input validated by schema; unknown fields rejected | Send extra fields and long strings |
| ☐ | All SQL uses parameters | Code search for string-built SQL: none |
| ☐ | Error answers contain no stack traces, SQL, or paths | Cause an error on staging and read the answer |
| ☐ | Security headers set; CORS allows only exact origins | Check response headers |
| ☐ | App user in the database cannot DROP or ALTER | Try it with the app credentials |
| ☐ | Webhooks verify signatures and ignore repeated events | Send a fake and a repeated webhook |
| ☐ | File uploads: type and size allow-list, random names, signed short-lived links | Try a `.exe` and a 50 MB file |
| ☐ | Dependabot, `npm audit`, CodeQL, and Trivy run in CI | CI results are green |
| ☐ | 2FA on GitHub, Cloudflare, Oracle, Firebase, and email accounts | Account settings |
| ☐ | Privacy policy is published; account deletion and data export work | Use the features yourself |
| ☐ | Incident plan is written and someone besides you has read it | Walk through Section 7.8 together |

## B. Reliability (no crash)

| | Check | How to verify |
|---|---|---|
| ☐ | Two API containers behind Caddy; one can fail without downtime | Stop api1: the site still works |
| ☐ | `restart: unless-stopped` on every container | Kill a container; it returns by itself |
| ☐ | `/health` and `/ready` work; Docker and Caddy use them | Stop PostgreSQL: `/ready` returns 503, `/health` stays 200 |
| ☐ | Timeouts set for HTTP, database, and Redis | Block Redis with a firewall rule: the API stays up |
| ☐ | Memory and CPU limits on every container; Node heap limit set | `docker stats` |
| ☐ | Graceful shutdown works (no dropped requests on deploy) | Run k6 while deploying |
| ☐ | Global error handler; process exits on `unhandledRejection` | Throw an error on staging |
| ☐ | Database pool size is below `max_connections` | Count: pool x containers |
| ☐ | Two Redis: cache with LRU, queue with `noeviction` | `redis-cli config get maxmemory-policy` on both |
| ☐ | Heavy tasks run in the queue; failed jobs reach a dead-letter list and alert | Force a failing job |
| ☐ | Docker logs rotate; disk alert at 80% | `docker inspect` shows log options |
| ☐ | Migrations are backward compatible and tested on a copy of production data | Staging run |
| ☐ | Nightly encrypted backup to a different provider; success is monitored | Check R2 and the monitor |
| ☐ | Restore drill done and timed in the last 30 days | Written result |
| ☐ | Rebuild guide tested: empty server to working system | Time it (target under 60 minutes) |
| ☐ | Image versions pinned; lock file used with `npm ci` | Search for `latest` tags |

## C. Speed (no delay)

| | Check | How to verify |
|---|---|---|
| ☐ | No sleeping or cold-start hosting for the main API | Wait 1 hour; first call is still fast |
| ☐ | p95 under 300 ms, cache hits under 20 ms on staging | Grafana latency panel |
| ☐ | Every WHERE/JOIN/ORDER BY column is indexed | `EXPLAIN ANALYZE` on every list query |
| ☐ | No N+1 queries on list endpoints | Count queries per request in a test |
| ☐ | All lists paginate with a maximum limit of 100 | Request `limit=10000`: gets capped |
| ☐ | Redis cache with TTL and invalidation on write | Change data and read it back |
| ☐ | Compression on; answers contain only needed fields | Check response size |
| ☐ | Public GET data cached at Cloudflare; private data sends `no-store` | Response headers |
| ☐ | Server region is close to most users | Latency test from a user's phone |
| ☐ | k6 test at 3x peak passed (errors < 1%, p95 < 300 ms) | Saved k6 report |
| ☐ | Slow-query log is on (200 ms) and reviewed weekly | Calendar reminder |
| ☐ | Argon2 hashing does not block other requests | Run login load test while reading other endpoints |

## D. Easy to manage

| | Check | How to verify |
|---|---|---|
| ☐ | One repo, one Compose file, one `make deploy` | A new person deploys with only the README |
| ☐ | Config comes from validated environment variables | Remove one variable: the app refuses to start |
| ☐ | CI runs tests, lint, type-check, and security scans on every pull request | CI page |
| ☐ | Staging exists and every release passes smoke tests there first | Pipeline log |
| ☐ | Deploy is a rolling restart with auto-rollback | Break the health check on purpose |
| ☐ | JSON logs with request-id in Loki; no secrets in logs | Search logs for a test password: none found |
| ☐ | Grafana dashboards and alerts exist and reach your phone | Send a test alert |
| ☐ | Outside uptime monitor (UptimeRobot) is on | Stop the API: you get a message |
| ☐ | OpenAPI file is generated and kept current | Compare with the real routes |
| ☐ | Runbook covers the top 9 problems | Section 13.3 copied and customized |
| ☐ | API versioning and the minimum-app-version setting work | Set a high minimum on staging: the app shows the update screen |
| ☐ | Admin actions are written to the audit log | Ban a test user and look in `audit_logs` |

## E. Flutter app

| | Check | How to verify |
|---|---|---|
| ☐ | Tokens only in `flutter_secure_storage` | Code search for `SharedPreferences` |
| ☐ | Dio interceptor refreshes once, queues parallel calls, and logs out on failure | Expire a token on purpose |
| ☐ | App sends `X-App-Version`; handles 426 with an update screen | Staging test |
| ☐ | App handles 400/422, 401, 403, 404, 409, 429, and 5xx with clear messages | Test each code with a mock |
| ☐ | Retry with back-off and jitter for network errors only; `Idempotency-Key` on create/pay calls | Turn on airplane mode while creating |
| ☐ | Pagination and cancel tokens used on lists and search | Scroll and search quickly |
| ☐ | Release build uses `--obfuscate` and keeps symbol files | Build log |
| ☐ | No API secrets, private keys, or passwords inside the app | Unzip the APK and search |
| ☐ | Firebase App Check is on and the server verifies the token | Call the API from a script: it is rejected |
| ☐ | Crashlytics is on; no personal data in logs or reports | Review a test crash report |
| ☐ | Push token is registered on login and removed on logout | Log out; no push arrives |
| ☐ | Store requirements met: privacy policy, data safety form, in-app account deletion | Store checklists |

### F. Launch day

| | Check | How to verify |
|---|---|---|
| ☐ | Restore from backup done once on a different machine | Written timing |
| ☐ | Cloudflare in “Full (strict)” SSL mode; origin firewall locked to Cloudflare | Test direct-IP access: it fails |
| ☐ | All default passwords and test accounts removed | Search the database |
| ☐ | Debug mode, Swagger UI, and `/metrics` are not public | Try the URLs from outside |
| ☐ | Alerts tested end to end; the right people receive them | Test alert |
| ☐ | Rollback tested on staging | Pipeline log |
| ☐ | Soft launch to a small group first (for example 5% of users) | Store staged rollout |
| ☐ | Someone is available for the first 48 hours | Schedule |
| ☐ | Status page or social post ready in case of an outage | Draft message saved |
| ☐ | Post-launch review booked for day 7 | Calendar |

# 16. A 30-day plan

| Week | Goal | Tasks |
|---|---|---|
| **1. Foundation** | A secure empty system that deploys itself | Create Oracle/other VM and Cloudflare. Harden the server. Docker Compose with Caddy, PostgreSQL, 2 Redis. CI pipeline with scans. Backup script and first restore test. |
| **2. Core features** | Auth and the first real endpoints | Register/login/refresh/logout with rotation. Users, devices, and your main data endpoints. Validation, error format, rate limits. OpenAPI. Connect the Flutter app with the Dio interceptor. |
| **3. Hardening** | Speed, reliability, and visibility | Caching, indexes, job queue, push messages. Health checks, rolling deploy, rollback. Prometheus, Grafana, Loki, Sentry, UptimeRobot, alerts. |
| **4. Prove it** | Test, fix, launch softly | Security manual tests, ZAP scan, k6 at 3x peak. Restore drill and rebuild drill. Close the master checklist. Soft launch, then watch the dashboards. |

## Sources and notes

Free-tier facts were checked in November 2026 and can change at any time. Please confirm them on the official pages:

- Render: free web services spin down after 15 minutes without traffic and take roughly a minute to wake (render.com/docs/free and render.com/articles).
- Oracle Cloud Always Free resources (docs.oracle.com, “Always Free Resources”) and the report on the June 2026 reduction of the free Arm allowance (InfoQ, July 2026).
- Cloudflare Workers limits and pricing (developers.cloudflare.com/workers/platform/limits and /pricing): free plan 100,000 requests per day, 10 ms CPU per request.
- OWASP API Security Top 10 (2023) (owasp.org/API-Security).
- Everything else in this guide (architecture, numbers such as timeouts and pool sizes, alert thresholds) is a recommended starting point. Measure your own app and adjust.